

ABSCHLIESSENDE ARBEIT
im Schuljahr 2025/26

Entwicklung eines kryptografisch sicheren und modernen Messenger-Dienstes

unter eigenständiger
Implementierung eines TLS-
ähnlichen Protokolls

vorgelegt von:
Anton Aparin

8C

unter Betreuung von:
Robin Gludovatz, BEd

Wien, am 13.2.2026

Gymnasium und Realgymnasium
der Vereinigung von Ordensschulen Österreichs

1180 Wien, Semperstraße 45
Schulkennzahl: 918016



Inhaltsverzeichnis

1. Abstract	6
2. Einleitung	7
2.1. Motivation	7
2.2. Zielsetzung	7
3. Grundlagen und Stand der Technik	8
3.1. Kryptografie	8
3.1.1. Hashfunktionen	8
3.1.2. Signaturalgorithmen	8
3.1.2.1. Grundprinzip	8
3.1.2.2. Ablauf	9
3.1.2.3. Typische Algorithmen	9
3.1.3. Schlüsselaustauschalgorithmen	9
3.1.3.1. Problemstellung	9
3.1.3.2. Idee	9
3.1.3.3. Moderne Varianten	10
3.1.4. Symmetrische Verschlüsselungsverfahren	10
3.1.4.1. Idee	10
3.1.4.2. AES (Advanced Encryption Standard)	10
3.1.4.3. AES-GCM (AES im Galois/Counter-Mode)	10
3.1.4.4. Anwendung	10
3.2. Angriffsmodelle	11
3.2.1. Man-in-the-Middle (MITM)	11
3.2.2. Replay-Attacken	11
3.2.3. Downgrade-Attacken	11
3.2.4. Key Compromise und Forward Secrecy	11
3.3. TLS im Überblick	11
3.3.1. Handshake	12
3.3.2. Datenübertragung	13
3.3.3. Sicherheitsmerkmale	13
4. Planung	14
4.1. Anforderungen	14
4.2. Sicherheitsziele	14
4.3. Abgrenzung	14
4.4. Technologiewahl	14

4.5.	Vorgehensweise	14
5.	Design	16
5.1.	Architekturübersicht	16
5.1.1.	Gesamtidee	16
5.1.2.	Komponenten	16
5.1.2.1.	Verwendete Technologien	16
5.1.3.	Kommunikationsfluss	16
5.2.	Protokollentwurf	17
5.3.	Sicherheitskonzept	19
6.	Umsetzung	20
6.1.	Protocol	20
6.1.1.	Handshake: SYN	20
6.1.2.	Handshake: ACK	21
6.1.3.	Nachrichtenübertragung: MSG	21
6.1.4.	Fehlerbehandlung	22
6.1.5.	Digest-Funktionen	23
6.1.6.	Zusammenfassung des Ablaufs	23
6.2.	Backend	23
6.2.1.	Authentifizierung	24
6.2.2.	Verwaltung der verbundenen Clients	24
6.2.3.	Weiterleitung von Nachrichten	24
6.2.4.	Start des Servers	25
6.3.	Frontend	26
6.3.1.	Interop zwischen Vue und WASM	26
6.3.2.	Zustandsaktualisierungen zwischen Vue und WASM	27
6.3.3.	Zustand in WASM	27
6.3.3.1.	Chat Zustand	29
6.3.4.	Logik im Frontend (WASM)	29
6.3.4.1.	<code>proxy_connect</code>	29
6.3.4.2.	<code>make_keypair</code>	30
6.3.4.3.	<code>read_packet</code>	30
6.3.4.4.	Verbindungsaufbau	31
6.3.4.4.1.	Handshake-Initialisierung (<code>init_chat</code>)	31
6.3.4.4.2.	Handshake-Verifikation und Handshake-Antwort (<code>handle_syn</code>)	32

6.3.4.4.3. Handshake-Verifikation (<code>handle_ack</code>)	33
6.3.4.5. Nachrichtenaustausch	33
6.3.4.5.1. Empfangen (<code>handle_encrypted_message</code>)	34
6.3.4.5.2. Senden (<code>send_message</code>)	35
6.3.4.5.3. Ablauf im Überblick	37
6.3.5. Frontend in Vue mit Vite	37
6.3.5.1. Projektstruktur	38
6.3.5.2. Reaktiver Zustand und Datenfluss	39
6.3.5.3. Aufbau und Verwaltung der WebSocket-Verbindung	40
6.3.5.4. Benutzeroberfläche und Komponentenstruktur	40
6.3.5.5. Fazit	41
6.4. Docker	41
6.4.1. Backend	42
6.4.2. Frontend & WebAssembly	42
6.4.3. Docker Compose	43
6.5. Paketablauf bei einer neuen Verbindung (Beispiel)	44
6.5.1. Empfang von Proxy	44
6.5.2. Authentifizierungsanfrage an den Proxy	44
6.5.3. SYN	45
6.5.4. ACK	45
6.5.5. EncryptedMessage	46
7. Test und Validierung mit Burp Suite	48
7.1. Testumgebung und Voraussetzungen	48
7.2. Aufbau des Tests	48
7.3. Normalablauf und Initialisierung	48
7.3.1. Simulierte unbeeinträchtigte Nutzersitzung	48
7.3.2. Start von Burp Suite und Aufnahme des Proxy-Verkehrs	49
7.3.3. Initialisierung: Schlüsselgenerierung und Proxy-Authentifizierung	49
7.3.4. Verbindungsaufbau zwischen Clients (SYN / ACK)	50
7.3.5. Laufende verschlüsselte Nachrichten (EncryptedMessage)	51
7.4. SYN im Namen eines fremden Clients senden	51
7.5. Rolle des Proxys und Anpassung für die Tests	53
7.6. Replay-Angriff auf SYN-Pakete	54
7.7. Replay-Angriff auf verschlüsselte Nachrichten	54
7.8. Manipulation des Handshakes (SYN/ACK)	55

7.9.	Unautorisierte Nachricht ohne Handshake	55
7.10.	Fazit	55
8.	Ergebnisse und Reflexion	57
8.1.	Zielsetzung	57
8.2.	Unterschiede zu TLS	57
8.2.1.	X.509-Zertifikate	57
8.2.2.	Beidseitige Authentifizierung	57
8.2.3.	Vereinfachter und minimalistisch gehaltener Handshake	57
8.3.	Persönliche Erkenntnisse	58
8.3.1.	Rust und Herausforderungen in der Implementierung	58
8.3.2.	Frontend-Integration und Ökosystem-Tools	58
8.3.3.	Tools zur Sicherheitsanalyse und Recherche	58
8.3.4.	Backend-Entwicklung	59
9.	Einsatzmöglichkeiten und Ausblick	60
10.	Anhang	61
	Literaturverzeichnis	62
	Abbildungsverzeichnis	63

1. Abstract

Diese Abschlussarbeit beschäftigt sich mit der Entwicklung einer kryptografisch sicheren und modernen Messenger-App. Ziel der Arbeit ist es, ein eigenes, TLS-ähnliches Kommunikationsprotokoll umzusetzen, um die Funktionsweise moderner Sicherheitsprotokolle besser zu verstehen. Zentral sind dabei die Sicherheitsziele Vertraulichkeit, Integrität, Authentizität und Forward Secrecy.

Zu Beginn werden die theoretischen Grundlagen der Kryptografie erläutert, darunter Hashfunktionen, digitale Signaturen, Schlüsselaustauschverfahren sowie symmetrische Verschlüsselung. Darauf aufbauend wird das Transport-Layer-Security-Protokoll (TLS) analysiert und als Grundlage für ein vereinfachtes, speziell auf einen Messenger zugeschnittenes Protokoll genommen. Dieses verwendet moderne Algorithmen wie Ed25519, X25519 und AES-GCM.

Im nächsten Teil wird der Messenger vollständig implementiert. Die Architektur besteht aus einem Proxy-Backend, einem Web-Frontend sowie einer in WebAssembly ausgeführten kryptografischen Logik, die in Rust entwickelt wurde.

Abschließend wird das Protokoll in einem professionellen Penetration-Testing-Programm getestet und analysiert. Die Ergebnisse zeigen, dass das entwickelte System die definierten Sicherheitsziele erfüllt und einen funktionierenden, Ende-zu-Ende-verschlüsselten Nachrichtenaustausch ermöglicht.

2. Einleitung

2.1. Motivation

Digitale Kommunikation wurde zu einem der wichtigsten Bestandteile unseres Lebens. Nachrichten werden konstant übermittelt, ob in Messengern, E-Mails oder Social Media. Jedoch findet diese Kommunikation oft über unsichere Netzwerke statt, wie zum Beispiel öffentliche Wi-Fi-Netzwerke, in denen die Informationen **gelesen und manipuliert** werden können.

Für den Schutz der Privatsphäre, Integrität und Authentizität der Daten sind deshalb kryptografische Algorithmen unverzichtbar.

Moderne Messenger wie Signal und WhatsApp setzen komplexe Sicherheitsprotokolle ein, die sichere, verschlüsselte Kanäle herstellen. Zentral ist dabei das **Transport-Layer-Security-Protokoll (TLS)**. Dieses ist ein Teil des bekannteren **Hypertext-Transfer-Protocol-Secure (HTTPS)** und wird in annähernd allen Internetanwendungen eingesetzt. Die Entwicklung solcher Protokolle ist jedoch technisch sehr kompliziert und bleibt auch für die meisten Software-Entwickler meistens eine „Blackbox“.

Aus diesem Grund bietet die Entwicklung eines eigenen Messengers auf Basis eines **TLS-ähnlichen Protokolls** eine gute Möglichkeit, verschiedene kryptografische Algorithmen praktisch anzuwenden und zu verstehen. Das Projekt ist gleichzeitig sehr herausfordernd, denn es erfordert sowohl tiefgehendes theoretisches Wissen als auch praktische Programmierfähigkeiten.

2.2. Zielsetzung

Das Ziel dieser Arbeit ist die Entwicklung eines TLS-ähnlichen Protokolls, das einen sicheren Kanal für die Kommunikation von zwei Partnern über ein unsicheres Netzwerk herstellen kann. Es soll anschließend in einem selbst entwickelten Messenger implementiert werden.

Unter Sicherheit wird verstanden, dass die Nachrichten

- geheim bleiben (**Vertraulichkeit**)
- nicht verändert werden können (**Integrität**)
- wirklich von der richtigen Person stammen (**Authentizität**)
- auch später mit gestohlenen Schlüsseln nicht entschlüsselt werden können (**Forward Secrecy**) [1]

3. Grundlagen und Stand der Technik

3.1. Kryptografie

In dieser Arbeit werden mehrere kryptografische Algorithmen für verschiedene Zwecke verwendet, zum Beispiel zum Signieren, Verschlüsseln, Schlüsselaustausch oder Hashen.

3.1.1. Hashfunktionen

Eine Hashfunktion ist ein Verfahren, das Text oder Bytes beliebiger Länge auf eine Ausgabe fester Länge abbildet. Diese Ausgabe bezeichnet man als Digest oder Hashwert. Solche Funktionen werden für Integritätsprüfungen, digitale Signaturen und Passwortspeicherungen verwendet. Ein Beispiel einer solchen Funktion ist **SHA256 (Secure Hash Algorithm 256)**. Sie nimmt eine beliebige Anzahl an Bytes ein und gibt genau 256 Bits (32 Bytes) zurück.

Eigenschaften:

1. **Determinismus:** Bei gleichem Input bekommt man immer denselben Output.
2. **Schnelle Berechenbarkeit.**
3. **Einwegfunktion:** Während es sehr einfach ist, den Digest eines Wertes zu berechnen, ist das Berechnen des Inputs aus dem Digest praktisch unmöglich¹.
4. **Kollisionsresistenz:** Es ist praktisch unmöglich, zwei verschiedene Eingaben zu finden, die denselben Digest erzeugen.
5. **Avalanche-Effekt:** Wenn ein einzelnes Bit der Eingabe geändert wird, wird eine völlig andere Ausgabe erzeugt. [2], [3]

3.1.2. Signaturalgorithmen

3.1.2.1. Grundprinzip

Wenn man eine Nachricht im Internet verschickt, will man nicht nur, dass sie geheim bleibt, sondern auch, dass der Empfänger sicherstellen kann, dass sie von der richtigen Person kommt (**Authentizität**) und auf dem Weg nicht verändert wurde (**Integrität**).

Digitale Signaturen basieren auf asymmetrischer Kryptografie. Das heißt, dass jeder Benutzer

- einen **geheimen Schlüssel** (besitzt nur der Sender)
- einen dazugehörigen **öffentlichen Schlüssel** (jeder hat Zugriff)

¹Damit ist gemeint, dass die Rechnung auch auf den schnellsten Computern *sehr* lange dauern wird.

besitzt. Der Sender erstellt eine Signatur über eine Nachricht und verwendet dabei den geheimen Schlüssel. Der Empfänger kann dann mit dem öffentlichen Schlüssel des Senders prüfen, ob die Nachricht tatsächlich exakt so von dem angegebenen Sender gesendet wurde.

3.1.2.2. Ablauf

1. Der Sender macht die Nachricht bereit.
2. Der Sender berechnet den Hash über die Nachricht.
3. Der Sender verschlüsselt den Hash mit dem geheimen Schlüssel. Das wird dann zur Signatur.
4. Der Sender schickt sowohl die Nachricht als auch die Signatur an den Empfänger.
5. Der Empfänger berechnet ebenfalls den Digest der Nachricht.
6. Der Empfänger entschlüsselt die Signatur und vergleicht sie mit dem berechneten Hash.
7. Sind sie gleich, dann wurde bestätigt, dass die Nachricht tatsächlich unverändert vom richtigen Sender kommt. [1]

3.1.2.3. Typische Algorithmen

- **RSA-Signatur:** Basiert auf Primzahlen, ist aber nicht modern und relativ langsam.
- **DSA (Digital Signature Algorithm):** Wurde speziell für das Signieren entwickelt.
- **ECDSA:** Wie DSA, basiert aber auf elliptischen Kurven.
- **EdDSA (z. B. Ed25519):** Moderne Variante, basiert auch auf elliptischen Kurven, sehr sicher und effizient. Daher wird er heutzutage überall (und in dieser Arbeit) oft genutzt. [2]

3.1.3. Schlüsselaustauschalgorithmen

3.1.3.1. Problemstellung

Wenn zwei Personen im Internet geheim kommunizieren wollen, brauchen beide **denselben geheimen Schlüssel**. Aber wie können sie diesen über das Internet austauschen, ohne dass ihn jemand, der zuhört, diesen abhören kann?

Das lösen **Schlüsselaustauschalgorithmen**.

3.1.3.2. Idee

1. Jeder Teilnehmer generiert einen geheimen Schlüssel.
2. Jeder Teilnehmer berechnet den öffentlichen Schlüssel aus dem geheimen Schlüssel.

3. Die öffentlichen Schlüssel werden ausgetauscht.
4. Danach können beide Teilnehmer denselben Wert berechnen, aber ein Angreifer, der keine geheimen Schlüssel besitzt, kommt nicht an den Wert.
5. Aus diesem Wert wird der *Cipher*, der geheime asymmetrische Schlüssel, erstellt.

3.1.3.3. Moderne Varianten

- **ECDH (Elliptic Curve Diffie-Hellman)**.
- **X25519**: Moderner als ECDH, wird heutzutage überall verwendet (z. B. in TLS 1.3, Signal, WireGuard und in dieser Arbeit).

3.1.4. Symmetrische Verschlüsselungsverfahren

3.1.4.1. Idee

Nach einem Schlüsselaustausch haben beide Partner denselben Schlüssel und wollen Daten verschlüsseln und entschlüsseln.

Dafür wird meistens **AES (Advanced Encryption Standard)** verwendet. Es ist der weltweite Standard, der sehr effizient und zuverlässig ist.

3.1.4.2. AES (Advanced Encryption Standard)

1. AES teilt Daten in Blöcke von **128 Bit (16 Byte)** auf.
2. Jeder Block wird einzeln mit einem **Schlüssel (128, 192 oder 256 Bit)** verschlüsselt.
3. AES vermischt die Daten mit dem Schlüssel in mehreren Runden.
4. Am Ende erhält man zufällig aussehende Bytes, die nur mit demselben Schlüssel entschlüsselt werden können.

3.1.4.3. AES-GCM (AES im Galois/Counter-Mode)

AES ist nur für die Verschlüsselung verantwortlich. In der Praxis braucht man auch noch Integritätsschutz, damit man prüfen kann, ob die Nachricht verändert wurde.

Dafür nutzt man Betriebsmodi wie **GCM (Galois/Counter-Mode)**. Dabei wird zusätzlich ein öffentlicher **Prüfwert (Tag/Nonce)** geschickt, womit die **Integrität** geprüft werden kann.

3.1.4.4. Anwendung

Symmetrische Verschlüsselung wird überall angewendet, wo Sicherheit und Effizienz gefragt sind. Dazu gehören **TLS 1.3** – also **HTTPS** – **moderne Apps** wie Signal oder WhatsApp und **diese Arbeit**.

3.2. Angriffsmodelle

Ein gutes Protokoll muss verschiedene Attacken berücksichtigen, die auf einem öffentlichen Netz auftreten können. Folgend werden die relevanten Attacken aufgezählt.

3.2.1. Man-in-the-Middle (MITM)

Bei **MITM** dringt ein Angreifer zwischen den Client und den Server. Er kann alle Nachrichten, die gesendet werden, lesen, nach Bedarf verändern und neue Nachrichten erzeugen.

Das kann zum Beispiel vorkommen, wenn ein Angreifer auf einem öffentlichen Wi-Fi-Netz den Router imitiert und alle Verbindungen im Netz zuerst durch sich laufen lässt.

3.2.2. Replay-Attacken

Bei **Replay-Angriffen** wiederholt ein Angreifer bereits gesendete Nachrichten, sodass sie bei dem Empfänger doppelt ankommen. Das kann dazu führen, dass Befehle (wie Banktransaktionen) **doppelt ausgeführt werden**.

3.2.3. Downgrade-Attacken

Ein **Downgrade-Angriff** versucht, die verwendeten Algorithmen auf eine ältere, **unsichere Version** zu zwingen, um später einfacher an die Schlüssel zu kommen.

3.2.4. Key Compromise und Forward Secrecy

Wenn ein Schlüssel in der Zukunft kompromittiert wird, kann es einem Angreifer erlauben, alle verschlüsselten Nachrichten zu lesen.

Das kann mit **RSA (Rivest-Shamir-Adleman** – ein alter Algorithmus für asymmetrische Kryptografie) auftreten.

Moderne Protokolle können diese Art von Angriffen verhindern, indem sie ephemere, kurzlebige Schlüssel verwenden. Das Konzept wird **Forward Secrecy** genannt. [4]

3.3. TLS im Überblick

TLS ist der heutige Standard, um Daten sicher im Internet zu übertragen.

Mit den obengenannten Algorithmen stellt TLS **Vertraulichkeit, Integrität und Authentizität** sicher.

Um dies zu erreichen, wird TLS in zwei Phasen gegliedert: **Handshake** und **Datenübertragung**.

3.3.1. Handshake

Der TLS-Handshake passiert erst nach dem TCP-Handshake. So kann beispielsweise ein TLS 1.2 Handshake verlaufen:

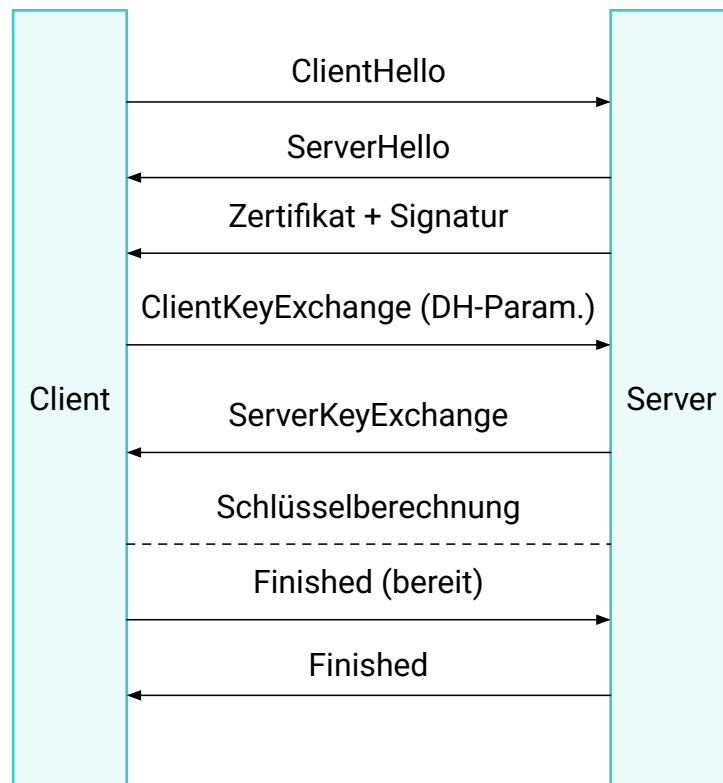


Abbildung 1: TLS 1.2 Handshake

1. **ClientHello**: Der Client meldet sich mit seiner unterstützten TLS-Version, einem Zufallswert (**Client-Random**) und einer Liste von möglichen Cipher-Suites.
2. **ServerHello**: Der Server antwortet mit seiner ausgewählten Cipher-Suite, einem eigenen Zufallswert (**Server-Random**) und seinem Zertifikat.
3. **Server-Signatur**: Der Server signiert die bisherigen Nachrichten, damit der Client die Echtheit prüfen kann.
4. **Signaturprüfung**: Der Client überprüft das Zertifikat und die Signatur → Authentizität ist gewährleistet.
5. **Schlüsselaustausch**: Der Client sendet seine Diffie-Hellman-Parameter, der Server seine. Beide können nun unabhängig voneinander den Premaster-Secret berechnen.
6. **Session Keys**: Aus Premaster-Secret, Client-Random und Server-Random werden symmetrische Schlüssel generiert.
7. **Finished-Nachrichten**: Beide bestätigen, dass sie denselben Schlüssel besitzen und nun mit verschlüsselter Kommunikation fortfahren können.

8. Ab jetzt: Die Datenübertragung läuft symmetrisch verschlüsselt (z. B. mit AES-GCM). [1]

3.3.2. Datenübertragung

- Nachrichten werden mit dem symmetrischen Schlüssel verschlüsselt/entschlüsselt.
- Jede Nachricht enthält eine integrierte Authentifizierung, wie z. B. ein Nonce in AES-GCM.
- Zähler und Nonces verhindern, dass alte Nachrichten erneut wiedergespielt werden (Replay-Schutz).

3.3.3. Sicherheitsmerkmale

- **Forward-Secrecy:** Auch wenn ein geheimer Schlüssel zukünftig kompromittiert wird, können die Nachrichten nicht entschlüsselt werden, da für jede Verbindung ein eigener symmetrischer Schlüssel verwendet wird.
- **Downgrade-Schutz:** Angreifer können die Verbindung nicht auf unsichere Algorithmen zwingen.

4. Planung

Als Erstes ist es wichtig, zu definieren, welche Funktionen die Chat-App haben soll, welche Sicherheitsziele die App anstreben soll, was nicht Teil der App ist und welche Technologien am besten zu diesen Zielen passen.

4.1. Anforderungen

Die Kernfunktionen der Anwendung sind:

- Sichere **Ende-zu-Ende-Verschlüsselung** von Nachrichten.
- **Sicherer Verbindungsaufbau** (Handshake) zwischen zwei Kommunikationspartnern.
- Ein minimales, aber funktionierendes **Frontend**, ähnlich zu Telegram.
- Ein **Proxy-Server**, der die Nachrichten weiterleitet, aber keinerlei die Nachrichten auf ihrem Weg entschlüsseln oder ändern kann.

4.2. Sicherheitsziele

Das Protokoll soll dieselben Sicherheitsziele wie TLS verfolgen, also **Vertraulichkeit, Integrität, Authentizität und Forward Secrecy**.

4.3. Abgrenzung

Diese Aspekte werden aus dem Projekt bewusst ausgeklammert:

- Ein **Zertifikatssystem** wie bei TLS.
- Schutz gegen **Denial-of-Service-Angriffe (DOS)**.

4.4. Technologiewahl

Zur Erreichung der Sicherheitsziele und Anforderungen wurden folgende Technologien/Sprachen ausgewählt:

- Die Programmiersprache **Rust** für das Backend und die Kryptografie-Logik. Sie ist eine der sichersten und modernsten auf der Welt und hat ein sehr praktisches Typ-System, das den Entwicklungsprozess beschleunigt und Fehler minimiert.
- **WASM (Web-Assembly)** für Krypto-Logik im Frontend, wobei der Code direkt aus **Rust** kompiliert wird.
- **Vue** als Frontend-Framework: Einfaches UI, schnelle Entwicklung.
- Ein **eigenes Protokoll** statt TLS für volle Kontrolle und als Lernebene.

4.5. Vorgehensweise

- Entwurf der **Projektstruktur**
- Modellierung des **Kommunikationsflusses**

- **Protokolldesign** und Festlegung der Abläufe
- **Implementierung** aller Komponenten
- Durchführung von **Tests**
- **Reflexion** der Ergebnisse und Auswertung

5. Design

5.1. Architekturübersicht

5.1.1. Gesamtidee

Die Chat-App besteht aus einem Frontend mit einer Benutzeroberfläche (UI), die direkt vom User bedient wird, sowie einem Backend als zentraler Proxy. Dieser Proxy ist dafür verantwortlich, Nachrichten an den richtigen Empfänger weiterzuleiten. Direkte Verbindungen zwischen den Nutzern existieren nicht.

5.1.2. Komponenten

Der Code ist in 4 Komponenten unterteilt:

- **Backend:** Code, der auf einem Server läuft. Ist dafür verantwortlich, dass Nachrichten an die richtige Person weitergeleitet werden. Kann die Nachrichten nicht lesen, sondern leitet sie nur blind weiter.
- **Frontend-WASM:** Enthält die kryptografische Logik. Hier werden Nachrichten verschlüsselt, entschlüsselt und die einzelnen Protokollschritte ausgeführt.
- **Frontend-Vue:** Enthält das Design der Website und dient als Schnittstelle zwischen dem User und der Logik in **Frontend-WASM**.
- **Protocol:** Definitionen für das Protokoll, damit jeder Benutzer und der Server die Nachrichten einheitlich serialisieren.

5.1.2.1. Verwendete Technologien

- **Rust:** Eine moderne Systemprogrammiersprache mit Fokus auf Sicherheit und Kontrolle. Sie eignet sich besonders gut für Kryptographie, da moderne Algorithmen gut unterstützt sind und alle auftretenden Fehler explizit behandelt werden. [5]
- **WebAssembly (WASM):** Ein standardisiertes Bytecode-Format, das direkt im Browser ohne Sicherheitsrisiken ausgeführt werden kann. Damit kann man Rust-Code im Browser ausführen. In diesem Projekt übernimmt das kompilierte WASM die kryptografische Logik, sodass auch sie sicher und schnell ablaufen kann. [6]
- **Vue.js:** Ein **JavaScript-Framework** zur Entwicklung reaktiver Benutzeroberflächen. Vue zeigt das UI an, stellt eine Verbindung zum Proxy her und ruft bei Bedarf Funktionen aus WASM. [7]

5.1.3. Kommunikationsfluss

Wie bereits erwähnt, findet keine direkte Verbindung zwischen den Clients statt. Stattdessen erfolgt der gesamte Nachrichtenverkehr über den Proxy.

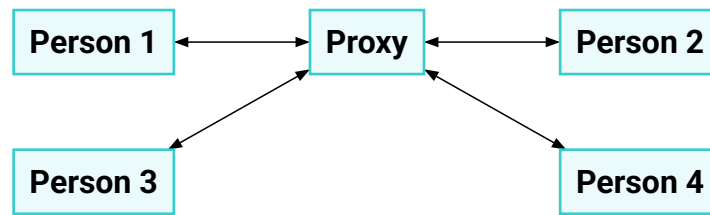


Abbildung 2: Kommunikationsfluss Gesamtbild

Da die Herstellung einer Web-Socket-Verbindung direkt aus Web-Assembly technisch zu komplex wäre, übernimmt **Frontend-Vue** diese Aufgabe. Die in **Frontend-WASM** implementierte Kryptografie wird als *Bibliothek* eingesetzt.

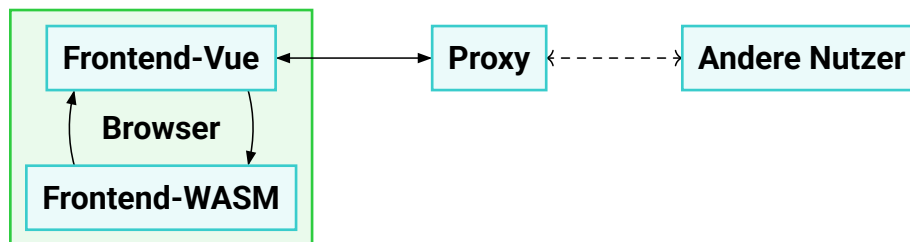


Abbildung 3: Kommunikationsfluss Frontend

5.2. Protokollentwurf

Als Grundlage für das Protokoll dient TLS 1.2 (vgl. Abbildung 1). TLS 1.3 wäre als Basis weniger optimal, da es wesentlich komplexer ist und die Adaptierung erschwert.

Damit das Protokoll zu einer Chat-App passt, wurden einige Änderungen vorgenommen:

1. Statt komplexer Zertifikate werden pure Ed25519 asymmetrische Schlüssel verwendet.
2. Beide Kommunikationspartner müssen sich gegenseitig authentifizieren. In TLS ist das optional und wird in der Praxis nur selten verwendet.
3. Da dieselbe Protokollversion und dieselben Cipher-Suits garantiert sind, müssen die Protokollversion und die Cipher-Suits nicht ausgetauscht werden. Der Handshake kann dadurch auf nur ein paar Schritte komprimiert werden.

Aus diesen Überlegungen entstand das folgende Protokoll. Dabei haben die Rollen **Client** und **Server** keine traditionelle Bedeutung, sondern nur, wer den Chat initiiert (Client) und wer darauf reagiert (Server). Die Namen SYN und ACK wurden bewusst aus TCP übernommen.

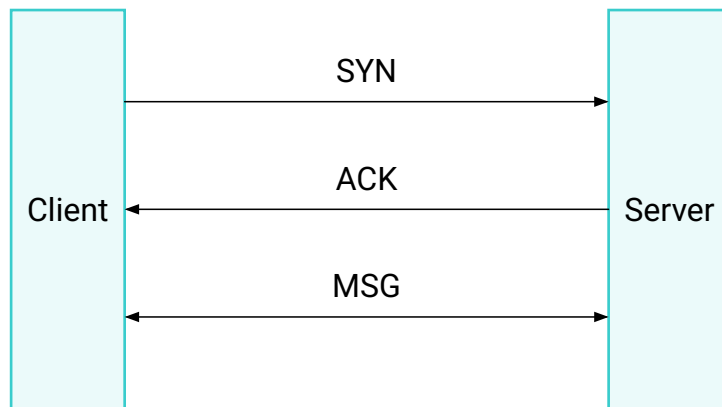


Abbildung 4: Protokollentwurf

1. Handshake-Initialisierung (Client → Server)

- Der Client erzeugt eine Chat-ID, Zufallswerte und ein ephemeres Schlüsselpaar.
- Der Client signiert die Anfrage (**SYN**) mit dem geheimen Schlüssel und sendet sie an den Server.

2. Handshake-Antwort (Server → Client)

- Der Server verifiziert die Signatur und Frische der Anfrage.
- Der Server erzeugt einen Zufallswert und ein ephemeres Schlüsselpaar.
- Der Server signiert die Antwort (**ACK**) und sendet sie.
- Der Server berechnet das geteilte Geheimnis und ist bereit, Nachrichten zu schicken.

3. Handshake-Verifikation

- Der Client verifiziert die Signatur der Antwort.
- Der Client berechnet das geteilte Geheimnis.

4. Sitzungsaufbau

- Beide Partner leiten einen symmetrischen Cipher (**AES-256-GCM**) aus dem gemeinsamen Geheimnis ab.
- Beide Partner speichern den Sitzungszustand.
- Die Sitzung ist nun verschlüsselt.

5. Nachrichtenaustausch

• Senden

- Der Sender signiert und verschlüsselt die Nachricht.
- Die Nachricht enthält die ID, Nonce und Signatur, sodass der Proxy die Nachricht auf Integrität und Authentizität prüfen kann, ohne sie zu entschlüsseln.

• Empfangen

- Der Empfänger prüft die Signatur und die Nachrichtenreihenfolge. Bei Fehlern wird die Nachricht abgelehnt.

- Der Empfänger entschlüsselt die Nachricht und aktualisiert den Zähler.

5.3. Sicherheitskonzept

Dieses Protokoll erreicht alle obengenannten Ziele (vgl. Abschnitt 2.2). Und zwar:

- **Vertraulichkeit:** Nach dem Schlüsselaustausch werden alle Nutzdaten mit einem gemeinsam abgeleiteten symmetrischen Sitzungsschlüssel (**AES-GCM**) verschlüsselt. Vor dem Abschluss des Handshakes werden nur Informationen übertragen, die nicht geheim sein müssen (z. B. öffentliche Schlüssel, Nonces, Zeitstempel).
- **Integrität:** Während des Handshakes werden Nachrichten mit dem eigenen geheimen Schlüssel signiert. Danach stellt **AES-GCM** sicher, dass jede nachträgliche Manipulation erkannt wird.
- **Authentizität:** Da die Signaturen überprüfbar sind, kann jede Partei sicherstellen, dass die empfangenen Nachrichten tatsächlich vom angegebenen Kommunikationspartner stammen. So wird ausgeschlossen, dass ein Angreifer gefälschte Nachrichten erfolgreich einschleusen kann.

6. Umsetzung

Im vorherigen Kapitel wurden die einzelnen Komponenten des Systems beschrieben. In diesem Kapitel wird die App praktisch umgesetzt. Der Fokus liegt dabei nicht auf der vollständigen Darstellung des Codes, sondern auf den wichtigsten Strukturen und Designentscheidungen.

Die Implementierung folgt der in Abschnitt 5.1.2 definierten Aufteilung in die vier Module: **Protocol**, **Backend**, **Frontend-WASM**, **Frontend-Vue**. Den Anfang macht das *Protocol*-Modul, das die Nachrichtentypen und deren Datenstrukturen definiert. Danach folgt das Backend, das die Nachrichten weiterleitet, und das Frontend-WASM, in dem die Logik des Protokolls umgesetzt wird. Abschließend wird Frontend-Vue vorgestellt, das als Benutzeroberfläche dient.

Zunächst wird das *Protocol*-Modul, das die Nachrichtentypen und Datenstrukturen definiert, implementiert. Danach folgt das Backend und Frontend-WASM. Den Schluss macht Frontend-Vue, das die Benutzeroberfläche definiert.

6.1. Protocol

Alle Nachrichten sollen in **Rust-Strukturen** definiert werden. Serialisierung und Deserialisierung erfolgen mit `serde`, einer Rust-Library, um Nachrichten als JSON darzustellen. Jeder Chat hat eine eindeutige `id`, damit die Nachrichten eindeutig zugeordnet werden.

6.1.1. Handshake: SYN

Der Handshake beginnt mit einer Anfrage vom Client an den Server.

```
rust
1 pub struct SYN {
2     pub id: u64,
3     pub client_ver_key: ed25519_dalek::VerifyingKey,
4     pub server_ver_key: ed25519_dalek::VerifyingKey,
5     pub client_eph_pub_key: x25519_dalek::PublicKey,
6     pub client_random: [u8; 32],
7     pub timestamp: u64,
8     pub signature: ed25519_dalek::Signature,
9 }
```

Listing 1: SYN Struktur

Enthaltene Daten:

- **Client Verifying Key**: fester öffentlicher Schlüssel für Signaturen.
- **Server Verifying Key**: öffentlicher Schlüssel des Servers.

- **Client Ephemeral Public Key:** öffentlicher Schlüssel für Diffie-Hellman-Schlüsselaustausch.
- **Client Random:** zufällige Bytes, schützen gegen Replay-Angriffe.
- **Timestamp:** zeitliche Gültigkeit, schützt gemeinsam mit Client-Random gegen Replay-Angriffe.
- **Signatur** über alle Felder, erstellt mit dem geheimen Schlüssel des Clients.

6.1.2. Handshake: ACK

Der Server antwortet mit einem Bestätigungspaket.

```
rust
1 pub struct ACK {
2     pub id: u64,
3     pub client_ver_key: ed25519_dalek::VerifyingKey,
4     pub server_ver_key: ed25519_dalek::VerifyingKey,
5     pub syn_digest: [u8; 32],
6     pub server_eph_pub_key: x25519_dalek::PublicKey,
7     pub server_random: [u8; 32],
8     pub signature: ed25519_dalek::Signature,
9 }
```

Listing 2: ACK Struktur

Enthaltene Daten:

- **Client und Server-Verifying-Keys:** Bestätigung der Identitäten.
- **Digest des SYN** stellt sicher, dass diese Antwort zum richtigen SYN gehört.
- **Server-Ephemeral-Public-Key.**
- **Server-Random.**
- **Signatur** über alle Felder, erstellt mit dem geheimen Schlüssel des Servers.

6.1.3. Nachrichtenübertragung: MSG

Nach Abschluss des Handshakes wird der gemeinsame Cipher berechnet. Nun können Textnachrichten verschickt werden.

rust

```

1 pub struct EncryptedMessage {
2     pub id: u64,
3     pub sender_ver_key: ed25519_dalek::VerifyingKey,
4     pub receiver_ver_key: ed25519_dalek::VerifyingKey,
5     pub message: EncryptedMessagePayload,
6     pub nonce: GenericArray<u8, U12>,
7     pub signature: ed25519_dalek::Signature,
8 }

```

Listing 3: EncryptedMessage Struktur

Enthaltene Daten:

- **Sender- und Empfänger-Keys** zur Authentifizierung mit dem Proxy.
- **Nonce**: Schützt AES-GCM vor Wiederholungen und bietet Nachweis für Integrität.
- **Ciphertext**: Verschlüsselte Nachricht
- **Signatur** über alle Felder

Dabei ist `EncryptedMessagePayload` eine Abstraktion über Bytes als Ciphertext (`Vec<u8>`).

Werden diese entschlüsselt, erhält man `DecryptedMessagePayload`:

rust

```

1 pub struct DecryptedMessagePayload {
2     pub id: u64,
3     pub content: String,
4 }

```

Listing 4: DecryptedMessagePayload Struktur

Es enthält:

- **ID**: Zähler der empfangenen Nachrichten. Schützt vor Replay-Angriffen und gibt die korrekte Reihenfolge der Nachrichten an.
- **Content**: Die Textnachricht selbst.

6.1.4. Fehlerbehandlung

Alle Strukturen haben `verify_signature()`-Methoden. Damit können Nachrichten sofort nach Integrität und Authentizität überprüft werden.

Fehler (falsche Signatur, ungültiges JSON, abgelaufene Pakete, usw.) werden explizit zurückgegeben. So, zum Beispiel, der Konstruktor von `EncryptedMessage`, der entweder sich selbst (`Self = EncryptedMessage`) konstruiert oder einen Fehler (`EncryptionError`) zurückgibt:

```

1 impl EncryptedMessage {
2     pub fn new(/* Felder */ ) → Result<Self, EncryptionError> { /* Code */ }
3
4     // Andere Methoden ...
5 }
6
7 pub enum EncryptionError {
8     #[error("Failed to serialize the message")]
9     SerializationFailed(#[from] serde_json::Error),
10
11     #[error("AES encryption failed")]
12     EncryptionFailed(String),
13 }

```

Listing 5: Fehlerbehandlung `EncryptedMessage` Beispiel

6.1.5. Digest-Funktionen

Einige Strukturen haben eigene `digest()`-Methoden. Diese erzeugen einen Hashwert über alle wichtigen Felder, außer den Digest selbst.

6.1.6. Zusammenfassung des Ablaufs

1. **SYN**: Client startet Handshake.
2. **ACK**: Server bestätigt Handshake.
3. Beide berechnen denselben geheimen Schlüssel.
4. **MSG**: Ab hier nur noch verschlüsselte Kommunikation.

6.2. Backend

Das Backend wurde ebenso in Rust geschrieben und übernimmt die Rolle eines Proxys. Es vermittelt die Nachrichten zwischen den Nutzern, hat aber selbst keine geheimen Schlüssel und kann somit die Nachrichten nicht entschlüsseln.

Der Proxy hat vor allem 4 Aufgaben:

1. Authentifizierung neuer Verbindungen.
2. Verwaltung aller verbundenen Clients.
3. Weiterleitung von Nachrichten.
4. Verifizierung der *Sender-Keys*.

6.2.1. Authentifizierung

Jeder Client muss sich beim Verbindungsaufbau mit dem **Verifying Key** und mit einer Signatur **ausweisen**. Dazu schickt der Proxy dem Client 32 zufällige Bytes, die der Client mit seinem Schlüssel signieren muss.

Der verwendete Datentyp sieht dabei so aus:

```
rust
1 pub struct AuthPacket {
2     pub ver_key: ed25519_dalek::VerifyingKey,
3     pub signature: ed25519_dalek::Signature,
4 }
5
6 impl AuthPacket {
7     pub fn verify(&self, random: &[u8; 32]) → Result<(),
8         ed25519_dalek::SignatureError> {
9         self.ver_key.verify_strict(random, &self.signature)
10    }
11 }
```

Listing 6: AuthPacket Struktur

Das Backend prüft schließlich, ob die Signatur korrekt ist. Ist sie inkorrekt, wird die Verbindung sofort abgebrochen.

6.2.2. Verwaltung der verbundenen Clients

Alle Clients werden in einer `HashMap` gespeichert. Als Schlüssel dienen die öffentlichen Schlüssel der Clients. Der Wert ist ein „Sender“, über den Nachrichten an diesen Client geschickt werden können.

```
rust
1 type Tx = UnboundedSender<Message>;
2 type PeerMap = Arc<Mutex<HashMap<ed25519_dalek::VerifyingKey, Tx>>>;
```

Listing 7: PeerMap Typdefinition

Dadurch wird sichergestellt, dass jede Nachricht an den richtigen Client gesendet wird.

6.2.3. Weiterleitung von Nachrichten

Wenn eine Nachricht am Proxy ankommt, wird geprüft:

1. Stimmt die Signatur?
2. Passt der Absender-Schlüssel zum angemeldeten Client?

Ist beides korrekt, wird der Empfänger in der `HashMap` gesucht und die Nachricht wird weitergeleitet.

```
rust
1 let recipient = match peers.iter().find(|(k, _)| *k == &receiver_ver_key) {
2     Some(kv) => kv,
3     None => {
4         warn!("Recipient {:?} not connected", receiver_ver_key);
5         return future::ok(());
6     }
7 };
8
9 if let Err(e) = recipient.1.unbounded_send(msg.clone()) {
10     error!("Failed to send message to {:?}: {:?}", receiver_ver_key, e);
11 }
```

Listing 8: Proxy Weiterleitung

6.2.4. Start des Servers

Die `main`-Funktion sorgt dafür, dass der Proxy gestartet wird. Er läuft unter der Adresse `127.0.0.1:8080`. Jede neue Verbindung wird asynchron aufgenommen und an die Funktion `handle_connection` weitergegeben:

```

1 #[tokio::main]
2 async fn main() → Result<(), IoError> {
3     tracing_subscriber::fmt::init();
4
5     let bind_addr = env::args()
6         .nth(1)
7         .unwrap_or_else(|| "127.0.0.1:8080".to_string());
8
9     let state = Arc::new(Mutex::new(HashMap::new()));
10    let listener = TcpListener::bind(&bind_addr).await?;
11    info!("Server listening on ws://{}", bind_addr);
12
13    while let Ok((stream, peer_addr)) = listener.accept().await {
14        let state = Arc::clone(&state);
15        tokio::spawn(handle_connection(state, stream, peer_addr));
16    }
17
18    Ok(())
19 }

```

Listing 9: Proxy `main`-Funktion

Somit kann der Proxy mehrere Clients gleichzeitig bedienen, da jede Verbindung asynchron verarbeitet wird.

6.3. Frontend

Das Frontend besteht aus zwei Teilen, die eng miteinander zusammenarbeiten:

- **WebAssembly (WASM)** beschäftigt sich mit den kryptographischen Operationen und mit der Paketlogik.
- **Vue** enthält die Benutzeroberfläche, hat eine Kopie des Zustands und stellt die Netzwerkverbindung mit dem Proxy über **WebSockets** [8] her.

6.3.1. Interop zwischen Vue und WASM

WebAssembly ist eine Zielplattform, in welche Rust, C++ oder Go kompiliert werden können. Somit kann Rust-Code im Browser bei nahezu nativer Geschwindigkeit ausgeführt werden.

Interop zwischen JavaScript und WASM geschieht über Bindings, die Funktionen aus Rust zu JavaScript exportieren. Somit kann der Code sinnvoll zwischen verschiedenen Sprachen (hier JS und Rust) aufgeteilt werden.

Rust exportiert Funktionen mithilfe von `wasm_bindgen`. Diese werden zu WASM kompiliert und dann in Vue importiert.

Beispielfunktion in Rust

```
rust
1 #[wasm_bindgen]
2 pub fn make_keypair() → Result<js_sys::Array, JsValue> {
3     // Beide Schlüssel generieren und in arr speichern
4
5     Ok(arr) // arr zurückgeben
6 }
```

Listing 10: Rust `#[wasm_bindgen]` Beispiel

Dann in JavaScript (vereinfacht)

```
js
1 import { make_keypair } from "frontend-wasm"; // make_keypair wird
  importiert
2
3 async generateKey() {
4     const result = await make_keypair(); // Rust-Funktion wird ausgeführt
5     const priv = Array.from(result[0]);
6     const pub = Array.from(result[1]);
7     this._storeKeypair(priv, pub); // Schlüssel wird gespeichert
8 }
```

Listing 11: JavaScript WASM Import Beispiel

6.3.2. Zustandsaktualisierungen zwischen Vue und WASM

Damit Vue nicht jedes Mal den kompletten Zustand des Chats an WASM weitergeben muss, ist es von Vorteil, dass beide Teile des Frontends eine Kopie des Zustands haben. Wenn zum Beispiel eine Nachricht ankommt, wird `read_packet` (später in Abschnitt 6.3.4 erklärt) gerufen. Der WASM-Teil liest die Nachricht, aktualisiert den eigenen Zustand und gibt Vue auch diese Aktualisierung zurück.

6.3.3. Zustand in WASM

In WASM wird der Zustand in einer Variable `STATE` der Struktur `MyState` als `RefCell` gespeichert. `RefCell` verhindert, dass der Zustand gleichzeitig von zwei Funktionen gelesen/verändert werden kann, und ist notwendig, damit der Code kompiliert werden kann.

rust

```

1  thread_local! {
2      static STATE: RefCell<MyState> = RefCell::new(MyState::default());
3  }
4
5  struct MyState {
6      pub sign_key: Option<ed25519_dalek::SigningKey>,
7      pub ver_key: Option<ed25519_dalek::VerifyingKey>,
8      pub chats: HashMap<u64, Chat>,
9      pub seen_nonces: HashSet<GenericArray<u8, U12>>,
10 }

```

Listing 12: Frontend-WASM Zustand

Der Zustand enthält

- Die Schlüssel
- Eine `HashMap` über Chat-IDs zu `Chat` Strukturen
- Ein `HashSet` über gesehene Nonces; schützt gegen Replay-Angriffe

Um auf den Zustand innerhalb der Funktionen zuzugreifen, muss am Anfang der Funktionen immer `STATE.with` gefolgt von einem `.borrow()` gerufen werden.

Zum Beispiel:

rust

```

1  #[wasm_bindgen]
2  pub fn proxy_connect(/* Felder */ → Result<String, JsValue> {
3      STATE.with(|state| {
4          let state = state.borrow();
5          let sign_key = state
6              .sign_key
7              .as_ref()
8              .ok_or_else(|| JsValue::from_str("No signing key available"))?;
9          let ver_key = state
10             .ver_key
11             .as_ref()
12             .ok_or_else(|| JsValue::from_str("No verifying key
available"))?;
13
14             // Code
15     })
16 }

```

Listing 13: Zustand Zugriff Beispiel

Wird `STATE` zu dieser Zeit verwendet, so wartet `STATE.with` bis die andere Funktion den `STATE` loslässt.

6.3.3.1. Chat Zustand

Ein Chat hat zwei mögliche Zustände: `Syn` und `Encrypted`.

Im Zustand `syn` wurde der Chat bereits vom Client initiiert, das heißt, ein `syn`-Paket wurde gesendet, jedoch noch kein `ack`-Paket empfangen. Die Verbindung befindet sich somit im Aufbau.

Befindet sich der Chat im Zustand `Encrypted`, wurde der Handshake erfolgreich abgeschlossen, und es besteht eine vollständig etablierte und verschlüsselte Verbindung zwischen den Kommunikationspartnern.

6.3.4. Logik im Frontend (WASM)

Die Aufgabe des Frontends unterscheidet sich von der des Proxys drastisch. Während der Proxy nur die Signaturen prüft und Nachrichten weiterleitet, ist das Frontend sowohl für die korrekte Implementierung des Handshakes als auch für die Verschlüsselung verantwortlich.

Im WASM-Teil des Frontends werden alle Funktionen geschrieben, die Pakete lesen/erzeugen und Schlüssel generieren – also alles, was von Rust profitieren würde. Dazu gehören:

- `proxy_connect`: Signiert die zufälligen Bytes, um sich beim Proxy auszuweisen (siehe Abschnitt 6.2.1).
- `make_keypair`: Generiert ein Schlüsselpaar (öffentlicher und geheimer Schlüssel).
- `load_sign_key`: Lädt einen bereits generierten Schlüssel.
- `init_chat`: Beginnt ein Chat mit einem anderen Nutzer.
- `read_packet`: Liest ein einkommendes Paket und entscheidet, was damit gemacht werden soll.
- `send_message`: Schickt eine Nachricht an einen Nutzer, mit dem schon eine Verbindung besteht.

6.3.4.1. `proxy_connect`

Wenn eine Verbindung zum Proxy hergestellt wird, sendet der Proxy dem Client 32 zufällige Bytes (vgl. Abschnitt 6.2.1). Diese müssen mit dem geheimen Schlüssel vom Client signiert werden und gemeinsam mit dem öffentlichen Schlüssel an den Proxy zurückgesendet werden.

```

1 #[wasm_bindgen]
2 pub fn proxy_connect(proxy_random: Vec) → Result<String, JsValue> {
3     // STATE ausborgen
4
5     let mut sign_key = sign_key.clone();
6     let signature = sign_key.sign(&proxy_random);
7
8     let auth_packet = auth_packet::AuthPacket {
9         ver_key: *ver_key,
10        signature,
11    };
12
13    serde_json::to_string(&auth_packet)
14        .map_err(|e| JsValue::from_str(&format!("Failed to serialize auth
15        packet: {e}")))
15 }

```

Listing 14: `proxy_connect` Implementation

6.3.4.2. `make_keypair`

Hat ein Nutzer noch kein Schlüsselpaar, generiert diese Funktion ein zufälliges Paar.

6.3.4.3. `read_packet`

Sobald ein neues Paket ankommt, wird dessen Inhalt an die Funktion `read_packet` Funktion weitergeleitet. Sie dient als zentraler Dispatcher.

Abhängig vom Typ des Pakets werden interne Zustände aktualisiert, neue Nachrichten erzeugt und/oder Antwortpakete generiert. Das Ergebnis wird in einem `ReadPacketReturnValue` kodiert.

Behebbarer Fehler werden als spezielle Systemnachrichten gemeinsam mit anderen Nachrichten an das Frontend zurückgegeben. Unbehebbarer Fehler führen dazu, dass anstelle eines `ReadPacketReturnValue` ein `JsValue` mit einer Fehlermeldung als Text zurückgegeben wird.

rust

```

1 struct ReadPacketReturnValue {
2     message_updates: Vec<MessageUpdateToFrontend>,
3     packet: Option<String>,
4 }
5
6 pub fn read_packet(packet: String) → Result<String, JsValue>

```

Listing 15: `read_packet` Signatur

Der empfangene `packet`-String wird zunächst deserialisiert. Abhängig vom Ergebnis entsteht ein SYN, ACK oder EncryptedMessage-Paket.

Dieses wird anschließend an eine spezialisierte Handler-Funktion weitergeleitet:

- `handle_syn` (Initialisierung eines Chats)
- `handle_ack` (Bestätigung des Schlüsselaustauschs)
- `handle_encrypted_message` (ver-/entschlüsselte Kommunikation)

Alle Handler verwenden das gleiche Rückgabeformat wie `read_packet`, sodass die Verarbeitung einheitlich bleibt. Nachfolgend ein Beispiel einer Handler-Signatur:

rust

```

1 pub fn handle_syn(
2     packet: Box<protocol::SYN>,
3     state: &mut MyState,
4     sign_key: &mut ed25519_dalek::SigningKey,
5     ver_key: ed25519_dalek::VerifyingKey,
6 ) → Result<(Vec<MessageUpdateToFrontend>, Option<String>), JsValue>

```

Listing 16: `handle_syn` Funktionsignatur Beispiel

Der erfolgreiche Rückgabetyt (`Vec<MessageUpdateToFrontend>, Option<String>`) enthält:

- **Neue Nachrichten**, die vom Frontend angezeigt werden sollen.
- **Optional ein Antwortpaket**, das an den Proxy versendet wird.

6.3.4.4. Verbindungsaufbau

6.3.4.4.1. Handshake-Initialisierung (`init_chat`)

Will ein Nutzer eine aktive Verbindung zu einem anderen herstellen, wird diese Funktion gerufen. Sie nimmt als Argument den öffentlichen Schlüssel der anderen Person und gibt ein Paket zurück, das an den Proxy geschickt werden muss. Dies ist der erste Schritt bei der Herstellung einer sicheren Verbindung.

So läuft sie ab:

1. Ein `client_random` (32 zufällige Bytes) werden generiert.
2. Eine zufällige Chat-ID – 64-bit ganze positive Zahl (`u64`) – wird erstellt.
3. Ein ephemeres Schlüsselpaar wird generiert.
4. Es wird der jetzige Zeitstempel (Millisekunden seit 1970) genommen.
5. Nun kann das `SYN` Paket erstellt werden.

6.3.4.4.2. Handshake-Verifikation und Handshake-Antwort (`handle_syn`)

Initiiert eine andere Person – der Client – ein Chat mit `init_chat` (Abschnitt 6.3.4.4.1), welche dabei ein `SYN`-Paket absendet, wird bei dem Server die `handle_syn` Funktion mit dem Inhalt des Pakets gerufen.

Bevor ein Handshake bestätigt werden kann (siehe Abschnitt 5.2), werden die Felder des `SYN`-Pakets ausführlich validiert.

Dabei können behebbare Fehler auftreten, u. a.:

- Ein Chat mit derselben Chat-ID existiert bereits.
- Die Signatur des Pakets ist ungültig.
- Der öffentliche Schlüssel stimmt nicht mit dem erwarteten Schlüssel überein.
- Das Paket ist älter als 5 Sekunden.

Wenn keine dieser Fehlerbedingungen vorliegt, wird gemäß Abschnitt 5.2 ein `ACK`-Paket erzeugt und zurückgegeben.

Nach dem Empfang eines gültigen `SYN` generiert die Server-Seite ihr ephemeres Schlüsselpaar und kann das gemeinsame Sitzungsschlüsselmaterial sofort ableiten. Damit ist der Handshake aus Sicht des Servers bereits abgeschlossen und der Server ist **ab diesem Zeitpunkt vollständig kryptografisch abgesichert**.

Die Verbindung ist allerdings erst dann wechselseitig abgesichert, sobald der Client das `ACK` empfangen, überprüft und ebenfalls den gemeinsamen Sitzungsschlüssel berechnet hat. Erst anschließend können beide Seiten verschlüsselte Nachrichten über `MSG`-Pakete austauschen.

Durch die ausschließliche Verwendung ephemerer erzeugter Schlüsselmaterialien (X25519) erreicht das Protokoll Forward Secrecy: Kompromittierte Langzeitschlüssel führen **nicht** zur Entschlüsselung bereits übertragener Nachrichten.

6.3.4.4.3. Handshake-Verifikation (`handle_ack`)

Auf der Client-Seite stellt `handle_ack` den Abschluss des Handshakes dar. Sie wird aufgerufen, sobald der Client ein `ACK`-Paket vom Gesprächspartner erhält.

Zuerst werden die beim Versand des ursprünglichen `SYN` hinterlegten Daten aus dem lokalen Zustand rekonstruiert. Anschließend wird überprüft, ob das Paket inhaltlich konsistent ist. Fehlerhafte oder widersprüchliche `ACK`-Pakete werden als Systemmeldungen an das Frontend gemeldet, ohne dass weitere Aktionen ausgeführt werden.

Die Validierung umfasst dabei folgende Aspekte:

- Die Signatur des Pakets muss gültig sein.
- Der angegebene Empfänger-Schlüssel muss dem erwarteten öffentlichen Schlüssel entsprechen.
- Die im Paket enthaltene Prüfsumme des ursprünglichen `SYN` (Digest) darf nicht abweichen.
- Der im Handshake ausgehandelte Server-Schlüssel muss mit dem ursprünglich empfangenen übereinstimmen.

Sind alle Prüfungen erfolgreich, ist sichergestellt, dass:

- Keine Manipulation durch Dritte erfolgt ist.
- Beide Seiten denselben Handshake-Partner bestätigen.
- Das `ACK` tatsächlich auf das zuvor gesendete `SYN` referenziert.

Erst dann wird aus dem eigenen ephemeren Private Key und dem Public Key der Gegenseite ein gemeinsames Geheimnis berechnet. Daraus wird ein AES-256-GCM-Cipher für die weitere, verschlüsselte Kommunikation abgeleitet.

Zum Schluss wird der Chat-Status in `Chat::Encrypted` überführt, womit nun auch der Client die verschlüsselte Kommunikation aufnehmen kann. Der Handshake ist damit beidseitig abgeschlossen.

6.3.4.5. Nachrichtenaustausch

Nach dem Abschluss des Handshakes befinden sich beide Parteien in einem synchronisierten, verschlüsselten Zustand (`Chat::Encrypted`).

Ab diesem Zeitpunkt können Nachrichten in beide Richtungen verschlüsselt übertragen werden.

Das Frontend unterscheidet dabei zwischen eingehenden (`handle_encrypted_message`) und ausgehenden (`send_message`) Nachrichten.

Beide Funktionen greifen auf denselben AES-GCM-Cipher und denselben Sequenzzähler zu.

6.3.4.5.1. Empfangen (`handle_encrypted_message`)

Immer wenn ein verschlüsseltes Paket ankommt, ruft `read_packet` intern `handle_encrypted_message` auf. Diese Funktion prüft und entschlüsselt das Paket und gibt die resultierende Nachricht an das Frontend weiter.

Zuerst wird geprüft, ob der Chat überhaupt existiert und der Zustand `Encrypted` ist.

```
rust
1 if let Some(chat) = state.chats.get_mut(&packet.id) {
2     if let Chat::Encrypted { cipher, other_ver_key, .. } = chat {
3         // Nachricht entschlüsseln
4     } else {
5         // Chat existiert, ist aber nicht verschlüsselt
6     }
7 } else {
8     // Chat existiert nicht
9 }
```

Listing 17: `handle_encrypted_message` Prüfung Beispiel

Danach folgen Integritätsprüfungen:

- **Sequenz-ID:** verhindert doppelte oder übersprungene Nachrichten.
- **Absender-Schlüssel:** stellt sicher, dass das Paket tatsächlich vom Kommunikationspartner stammt.
- **AES-GCM-Entschlüsselung:** authentifiziert und entschlüsselt den Inhalt.
- **Nonce-Prüfung:** überprüft, ob der verwendete Nonce bereits früher empfangen wurde.

Der relevante Abschnitt:

rust

```

1 let message = match packet.message.decrypt(cipher, packet.nonce) {
2     Ok(m) => m,
3     Err(e) => {
4         message_updates.push(MessageUpdateToFrontend {
5             chat_id: packet.id.to_string(),
6             message: Message::System(format!(
7                 "decryption failed: {}", e
8             )),
9         });
10    return Ok((message_updates, None));
11 }
12 };

```

Listing 18: Nachrichtenentschlüsselung

Nach der Entschlüsselung wird die Nachricht intern gespeichert und an das Frontend weitergegeben:

rust

```

1 messages.push(Message::ToSelf(message.clone()));
2 message_updates.push(MessageUpdateToFrontend {
3     chat_id: packet.id.to_string(),
4     message: Message::ToSelf(message),
5 });

```

Listing 19: Einfügen und Weiterleiten einer empfangenen Nachricht

6.3.4.5.2. Senden (send_message)

Um eine Nachricht zu senden, ruft das Frontend die Funktion `send_message`.

Diese baut aus dem Text eine verschlüsselte Paketstruktur (`EncryptedMessage`) und gibt diese als serialisiertes JSON zurück, das dann an den Proxy gesendet wird.

Als Erstes wird der Nachrichten-Zähler erhöht und in der Payload gespeichert:

rust

```

1 let decrypted_payload = protocol::DecryptedMessagePayload {
2     id: *prev_id_self + 1,
3     content: message,
4 };
5 *prev_id_self += 1;

```

Listing 20: Zähler Erhöhung

Danach wird der Text mit AES-GCM verschlüsselt und signiert:

rust

```
1 let encrypted_packet = protocol::EncryptedMessage::new(  
2     chat_id,  
3     ver_key,  
4     *other_ver_key,  
5     &decrypted_payload,  
6     cipher,  
7     &mut sign_key.clone(),  
8 )?;
```

Listing 21: AES-GCM-Verschlüsselung

Abschließend wird das Paket serialisiert:

rust

```
1 let packet_json = serde_json::to_string(&packet)  
2     .map_err(|_| JsValue::from_str("failed to serialize packet"))?;
```

Listing 22: Paket-Serialisierung

Diese JSON-Darstellung wird an den Proxy übergeben und dann an den Empfänger weitergeleitet.

6.3.4.5.3. Ablauf im Überblick

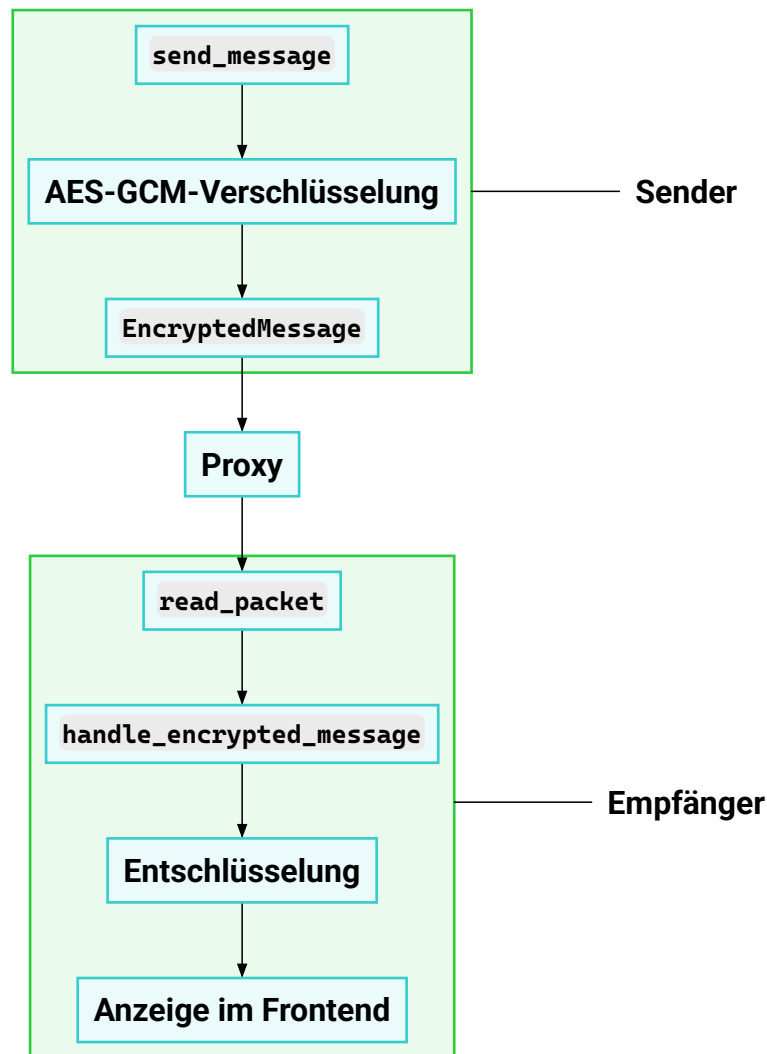


Abbildung 5: Senden Überblick

Damit ist der Ende-zu-Ende-verschlüsselte Nachrichtenaustausch vollständig implementiert: Jede Nachricht wird authentifiziert, nummeriert und verschlüsselt übertragen, womit die zentralen Sicherheitsziele von TLS erreicht werden.

6.3.5. Frontend in Vue mit Vite

Das Frontend bildet die Benutzerschnittstelle des Messengers und ist vollständig in **Vue 3** umgesetzt. Es ermöglicht die Interaktion mit dem Benutzer, die Anzeige von Chats und Nachrichten sowie die Steuerung kryptografischer Funktionen über eine WebAssembly-Schnittstelle. Das Build-System **Vite** wurde gewählt, um eine schnelle Entwicklungsumgebung mit integriertem Hot-Reloading, modularem Aufbau und optimierter Bereitstellung für Browser bereitzustellen. Durch die modulare Architektur und das reaktive Datenmodell von Vue kann die Benutzeroberfläche effizient auf Ände-

rungen des internen Zustands reagieren, ohne dass manuelle DOM-Manipulationen erforderlich sind.

6.3.5.1. Projektstruktur

Der Code des Frontends ist klar strukturiert. Der Einstiegspunkt `main.js` initialisiert die Anwendung und bindet die Hauptkomponente `App.vue` in das DOM ein. Diese Komponente ist ein Container für `ChatApp.vue`, welche die Benutzeroberfläche und Interaktionslogik lädt.

Die Datei `store.js` verwaltet den Anwendungszustand, baut die WebSocket-Verbindung auf und ist die Schnittstelle zu den WASM-Funktionen. `vite.config.js` enthält die Konfiguration des Build-Prozesses, während `style.css` die Stile der Seite bereitstellt.

Der Projektbaum ist wie folgt aufgebaut:

```
1 frontend-vue
2 | Dockerfile
3 | index.html
4 | LICENSE
5 | package.json
6 | package-lock.json
7 | README.md
8 | src
9 |   App.vue
10 |   assets
11 |   | vue.svg
12 |   components
13 |   | ChatApp.vue
14 |   main.js
15 |   store.js
16 |   style.css
17 | vite.config.js
```

6.3.5.2. Reaktiver Zustand und Datenfluss

Das Frontend nutzt die reaktiven Möglichkeiten von Vue 3, um Änderungen im Zustand automatisch in der Benutzeroberfläche zu reflektieren. Hier kommen die Funktionen `reactive`, `ref` und `computed` auf der Composition-API zum Einsatz.

Der zentrale Zustand wird in `store.js` mit `reactive()` definiert. Darin werden alle aktiven Chats, die aktuelle Chat-ID, der Authentifizierungsstatus, der eigene Schlüssel und der Status der WebSocket-Verbindung verwaltet.

```
js
1 const state = reactive({
2   chats: [], // [{ id, name, messages: [] }]
3   currentChatId: null,
4   signKey: null,
5   verKey: null,
6   websocket: null,
7   isKeyLoaded: false,
8   isAuthenticated: false,
9 });
10
11 export default {
12   state,
13
14   // Funktionen
15 }
```

Listing 23: `store.js` Zustand

Die Datenflüsse gehen in beide Richtungen:

1. Eine Benutzeraktion im Interface (z. B. das Senden einer Nachricht) ruft eine Methode im Store auf.
2. Der Store verarbeitet die Eingabe, verschlüsselt die Nachricht über die Rust-WASM-Bibliothek und sendet sie über den WebSocket an das Backend.
3. Sobald eine Antwort oder neue Nachricht eintrifft, wird diese über `read_packet()` entschlüsselt und in den reaktiven Zustand übernommen.
4. Vue aktualisiert die Oberfläche automatisch entsprechend dem neuen Zustand.

Auf diese Weise bleibt die Anwendung einfach nachvollziehbar, modular und robust gegenüber Nebenwirkungen.

6.3.5.3. Aufbau und Verwaltung der WebSocket-Verbindung

Die Kommunikation mit dem Rust-Backend erfolgt über eine persistente WebSocket-Verbindung, die in `store.js` initialisiert wird. Sobald der Benutzer eine Schlüsseldatei importiert oder ein neues Schlüsselpaar generiert, ruft der Store die Funktionen `_connectWebSocket()` auf. Diese stellt eine Verbindung zum Proxy-Server (während des Entwicklungsprozesses unter `ws://127.0.0.1:8080`) her und verwaltet eingehenden sowie ausgehenden Pakete.

Der Ablauf gliedert sich in mehrere Phasen:

1. **Verbindungsaufbau:** Beim Öffnen der Verbindung (`ws.onopen`) wird der Status auf „verbunden“ gesetzt.
2. **Authentifizierung:** Die erste empfangene Nachricht wird an die Funktion `proxy_connect()` in der WebAssembly-Bibliothek weitergegeben. Diese überprüft den öffentlichen Schlüssel des Clients und leitet den Handshake-Prozess ein.
3. **Nachrichtenübertragung:** Eingehende Datenpakete werden von `read_packet()` analysiert und entschlüsselt. Anschließend werden sie als Ereignisse in den Zustand übernommen.
4. **Fehlerbehandlung:** Bei Unterbrechungen oder Fehlern (`ws.onerror`, `ws.onclose`) wird der Zustand zurückgesetzt, und der Benutzer kann manuell eine neue Verbindung herstellen.

Durch die klare Trennung von Verbindung, Authentifizierung und Nachrichtenverarbeitung bleibt die WebSocket-Logik übersichtlich und effizient.

6.3.5.4. Benutzeroberfläche und Komponentenstruktur

Die Oberfläche ist in zwei Bereiche geteilt: eine **Sidebar** und den **Chatbereich**.

Die Sidebar zeigt alle verfügbaren Chats an und enthält Bedienelemente für das Erstellen neuer Chats, den Import von Schlüsseln und die Anzeige der eigenen Verifikationsinformationen. Der Hauptbereich stellt die Nachrichtenliste des aktuell ausgewählten Chats dar und enthält ein Eingabefeld zum Verfassen von Nachrichten.

Alle sichtbaren Elemente werden in `ChatApp.vue` definiert. Diese Komponente verwendet die `<script setup>`-Syntax, wodurch die reaktive Logik direkt innerhalb des Komponenten-Scopes beschrieben wird. Über `v-model` sind Eingabefelder an den Zustand des Stores gebunden, während Klick-Ereignisse Methoden wie `sendMessage()` oder `startChat()` auslösen.

Das Layout nutzt Flexbox-Strukturen und CSS-Variablen, um sich an unterschiedliche Bildschirmgrößen anzupassen. Für mobile Geräte kann die Sidebar über eine Burger-Schaltfläche ein- und ausgeblendet werden.

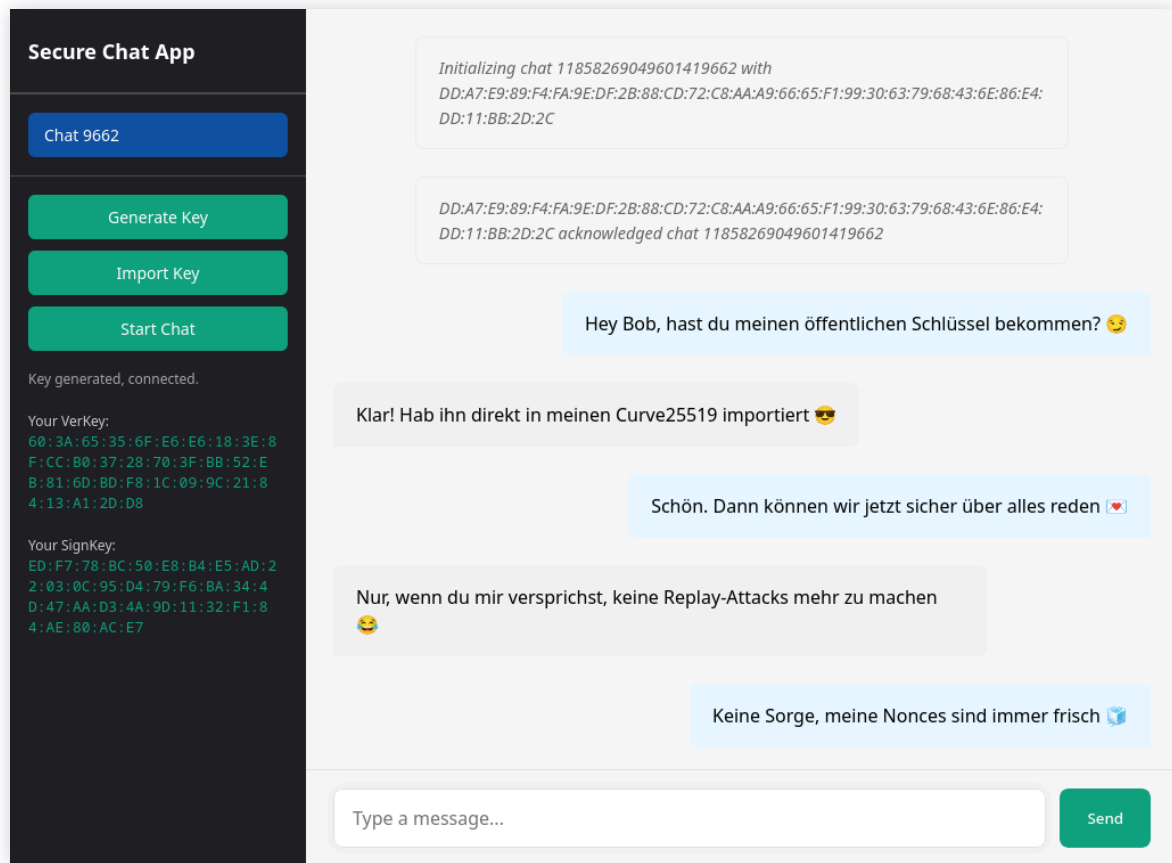


Abbildung 6: Screenshot der Chat-App

6.3.5.5. Fazit

Das Frontend zeigt, wie sich moderne Webtechnologien mit sicherer Systemprogrammierung kombinieren lassen. Vite ermöglicht eine schnelle und modulare Entwicklungsumgebung, während Vue 3 durch sein reaktives Datenmodell eine stabile und intuitive Benutzeroberfläche bereitstellt.

6.4. Docker

Für das Hosting des Frontends sowie des Backends wurde Docker verwendet. Damit kann die Anwendung isoliert und reproduzierbar ausgeführt werden. Es werden zwei Hauptkomponenten betrachtet: das Frontend, bestehend aus Vue, WASM und dem gemeinsamen Protokoll, und das Backend gemeinsam mit dem Protokoll. Jede Komponente wird in einem eigenen Container betrieben, wodurch die Abhängigkeiten und Build-Prozesse voneinander getrennt sind.

6.4.1. Backend

Das Backend ist eine Rust-Anwendung, die über Port 8080 erreichbar ist. Der Dockerfile für das Backend nutzt einen Multi-Stage-Build, um zunächst die Anwendung zu kompilieren und anschließend nur die fertige Binärdatei in das finale Image zu kopieren:

```
dockerfile
1 FROM rust:latest AS build
2 WORKDIR /app
3 COPY backend ./backend
4 COPY protocol ./protocol
5 WORKDIR /app/backend
6 RUN cargo build --release
7
8 FROM debian:bookworm-slim
9 WORKDIR /app
10 COPY --from=build /app/backend/target/release/backend /app/backend
11 EXPOSE 8080
12 CMD ["/app/backend"]
```

Listing 24: Backend Dockerfile

6.4.2. Frontend & WebAssembly

Das Frontend verwendet Vue mit Vite. Zusätzlich wird das WASM-Modul vor dem Vue-Build erstellt und in die Projektstruktur eingebunden. Auch hier wird ein Multi-Stage-Build verwendet:

```

1 # WASM-Build
2 FROM rustlang/rust:latest AS wasm-build
3 WORKDIR /app
4 RUN cargo install wasm-pack
5 COPY protocol ./protocol
6 COPY frontend-wasm ./frontend-wasm
7 WORKDIR /app/frontend-wasm
8 RUN wasm-pack build --target web --release
9
10 # Vue-Build
11 FROM node:22 AS frontend-build
12 WORKDIR /app
13 COPY frontend-vue ./frontend-vue
14 COPY --from=wasm-build /app/frontend-wasm/pkg ./frontend-wasm/pkg
15 WORKDIR /app/frontend-vue
16 RUN npm ci
17 RUN npm run build
18
19 # Nginx-Server
20 FROM nginx:stable-alpine
21 WORKDIR /usr/share/nginx/html
22 COPY --from=frontend-build /app/frontend-vue/dist ./
23 EXPOSE 80
24 CMD ["nginx", "-g", "daemon off;"]

```

Listing 25: Frontend Dockerfile

6.4.3. Docker Compose

Zur Koordination aller Container wird **docker-compose** verwendet. Dadurch können Backend und Frontend gleichzeitig gestartet werden:

```

1 services:
2   backend:
3     build: ./backend
4     ports:
5       - "8080:8080"
6   frontend:
7     build: ./frontend-vue
8     ports:
9       - "80:80"

```

Listing 26: docker-compose.yml

Durch Docker wird sichergestellt, dass die Anwendung auf jedem System gleich läuft, die Build-Abhängigkeiten korrekt ausgeführt werden und der Start einfach über `docker compose up` erfolgt.

6.5. Paketablauf bei einer neuen Verbindung (Beispiel)

Folgend ist ein Beispiel, das das Design des Protokolls abschließend veranschaulicht.

Es zeigt den vollständigen Nachrichtenfluss zwischen zwei Clients über den Proxy und veranschaulicht, wie die verschlüsselte Verbindung konkret erreicht wird.

Nach der Schlüsselerzeugung und der Authentifizierung beim Proxy erfolgt der eigentliche Aufbau einer neuen Verbindung zwischen zwei Kommunikationspartnern.

Folgend sind alle relevanten Pakete in der richtigen Reihenfolge dargestellt.

Jedes Paket wird tabellarisch beschrieben, wobei die linke Spalte die Feldnamen und die rechte Spalte die jeweiligen Eigenschaften oder Inhalte enthält.

6.5.1. Empfang von Proxy

Dieses Paket signalisiert, dass der Proxy bereit ist, und dient in erster Linie als Übertrag des öffentlichen Schlüssels bzw. als Initialisierungssignal.

Feld	Beschreibung
[32 Bytes]	<code>server_random</code> , zufälliger 32-Byte-Wert. Wird vom Client für die spätere Signaturprüfung genutzt. Nicht wiederverwendet.

Tabelle 1: Empfang von Proxy

6.5.2. Authentifizierungsanfrage an den Proxy

Der Benutzer authentifiziert sich, indem er die zufälligen 32 Bytes signiert und den eigenen öffentlichen Schlüssel mitsendet.

Feld	Beschreibung
ver_key	Öffentlicher Verifikationsschlüssel (32 Bytes), der den Client eindeutig identifiziert. Wird auch in späteren Paketen verwendet.
signature	64-Byte-Signatur über den <code>server_random</code> vom Proxy. Stellt sicher, dass der Client den zugehörigen privaten Schlüssel besitzt. Zufällig in ihrer Bitstruktur, aber deterministisch aus <code>server_random</code> und Schlüssel berechnet.

Tabelle 2: Authentifizierungsanfrage an den Proxy

6.5.3. SYN

Das SYN-Paket leitet den eigentlichen Verbindungsaufbau ein (analog zum TLS-Client-Hello). Es enthält sämtliche Parameter, die für die Aushandlung der Sitzungsschlüssel notwendig sind.

Feld	Beschreibung
id	64-Bit-Zufallszahl, die die Verbindung eindeutig identifiziert. Wird in allen folgenden Paketen wiederverwendet.
client_ver_key	Öffentlicher Verifikationsschlüssel des Clients (32 Bytes). Identisch mit dem Wert aus der Authentifizierung.
server_ver_key	Erwarteter öffentlicher Verifikationsschlüssel des Servers (32 Bytes). Dient der Zielbestimmung.
client_eph_pub_key	Ephemerer öffentlicher Schlüssel (32 Bytes). Nur für diese Verbindung gültig; ermöglicht Forward Secrecy.
client_random	32 zufällige Bytes, analog zum ClientRandom in TLS; dient zur Erzeugung des Sitzungsschlüssels.
timestamp	UNIX-Zeit in Millisekunden; schützt gegen Replay-Angriffe.
signature	64-Byte-Signatur über den gesamten Paketinhalt. Authentifiziert den Ursprung und schützt die Integrität.

Tabelle 3: SYN

6.5.4. ACK

Das ACK-Paket ist die Antwort des Servers auf das SYN. Es bestätigt die erhaltenen Parameter und liefert eigene Schlüsselanteile.

Feld	Beschreibung
id	Gleiche Sitzungs-ID wie im SYN (64 Bit). Verknüpft beide Pakete eindeutig.
client_ver_key	Öffentlicher Schlüssel des Clients; wiederholt zur Bindung der Nachricht an denselben Kommunikationspartner.
server_ver_key	Öffentlicher Schlüssel des Servers. Gleicher Wert wie im SYN angegeben.
syn_digest	32-Byte-Hashwert des empfangenen SYN-Pakets. Dient der Integritätsprüfung.
server_eph_pub_key	Ephemerer öffentlicher Schlüssel des Servers (32 Bytes).
server_random	32 zufällige Bytes, analog zum ServerRandom in TLS. Fließt ebenfalls in die Schlüsselableitung ein.
signature	64-Byte-Signatur über das ACK-Paket. Authentifiziert den Server.

Tabelle 4: ACK

6.5.5. EncryptedMessage

Nach erfolgreichem Handshake verfügen beide Partner über denselben Sitzungsschlüssel.

Ab diesem Punkt werden Nachrichten symmetrisch verschlüsselt übertragen.

Feld	Beschreibung
id	Gleiches Sitzungs-ID-Feld wie zuvor (64 Bit). Identifiziert den Nachrichtenkontext.
sender_ver_key	Öffentlicher Schlüssel des Absenders (32 Bytes). Dient der Zuordnung im Mehrbenutzersystem.
receiver_ver_key	Öffentlicher Schlüssel des Empfängers (32 Bytes). Entspricht dem Ziel der verschlüsselten Nachricht.
message	Verschlüsselter Nachrichteninhalt (z. B. 42 Bytes). Länge variabel, abhängig vom Klartext.
nonce	12-Byte-Zufallswert pro Nachricht. Gewährleistet Einmaligkeit im AES-GCM-Modus.
signature	64-Byte-Signatur über alle unverschlüsselten Felder. Wird nur vom Proxy kontrolliert

Tabelle 5: EncryptedMessage

7. Test und Validierung mit Burp Suite

In diesem Kapitel werden die Sicherheitsmaßnahmen der App verifiziert. Die Testumgebung und das Testverfahren werden beschrieben, mit denen die Sicherheit des Protokolls nachvollziehbar geprüft werden kann.

7.1. Testumgebung und Voraussetzungen

Die Tests wurden mit der **freien Edition von Burp Suite** durchgeführt. Es wurde der in Burp integrierte Browser verwendet. Dadurch erfolgt der Datenverkehr **nur über den Burp-Proxy**, der auf Port 8081 gestartet wird (Standard-Port 8080 wird bereits für die WebSocket-Kommunikation verwendet).

Es ist wichtig zu betonen, dass **nur ein Testnutzer** über den **Burp-Browser** angebunden ist; **Backend und der zweite Client** laufen in der Testumgebung **unverändert** und werden nicht durch Burp geleitet. So wird nur ein Nutzer getestet. Der Server und der andere Nutzer bleiben unbeeinträchtigt.

Das getestete Protokoll ist in **JSON** serialisiert. Dies erleichtert die Analyse mit Burp-Werkzeugen (Inspector, Repeater, Decoder), da auf Payloads direkt zugegriffen werden kann.

7.2. Aufbau des Tests

Nachdem der Normalablauf dokumentiert wurde, wird für jeden Test eine neue Verbindung hergestellt. Einzelne Pakete werden abgefangen, im **Burp-Repeater** modifiziert und anschließend erneut gesendet. Auf diese Weise können Sicherheitslücken gezielt gefunden werden. Die nachfolgende Reaktion von Client, Server und Proxy lässt klar wissen, ob der Angriff erfolgreich durchgeführt wurde.

7.3. Normalablauf und Initialisierung

Zu Beginn der Tests werden Backend und Frontend lokal gestartet. Dazu wird im Stammverzeichnis der Befehl `docker compose up -d` ausgeführt; nach erfolgreichem Start ist die Anwendung über `http://localhost:80` bzw. `http://127.0.0.1` erreichbar. Dieser Schritt stellt die Baseline-Umgebung her, in der Server- und Proxy-Instanzen in der erwarteten, unveränderten Konfiguration laufen.

7.3.1. Simulierte unbeeinträchtigte Nutzersitzung

Zur Simulation eines unbeeinträchtigten Kommunikationspartners wird die Weboberfläche der Applikation in einem gewöhnlichen Browser (z. B. Mozilla Firefox) geöffnet. Dieser Nutzer agiert ohne Einbindung in Burp.

7.3.2. Start von Burp Suite und Aufnahme des Proxy-Verkehrs

Anschließend wird Burp Suite gestartet. Im Bereich „Proxy“ des Programms wird der integrierte Burp-Browser geöffnet und der Intercept-Modus aktiviert, sodass der gesamte vom Burp-Browser initiierte Verkehr über den Burp-Proxy auf Port 8081 geleitet und dort abgefangen werden kann. Die in Burp aufgezeichneten Verkehrseinträge werden in der **History** protokolliert und können in weiteren Untersuchungen in den Werkzeugen Repeater verwendet werden.

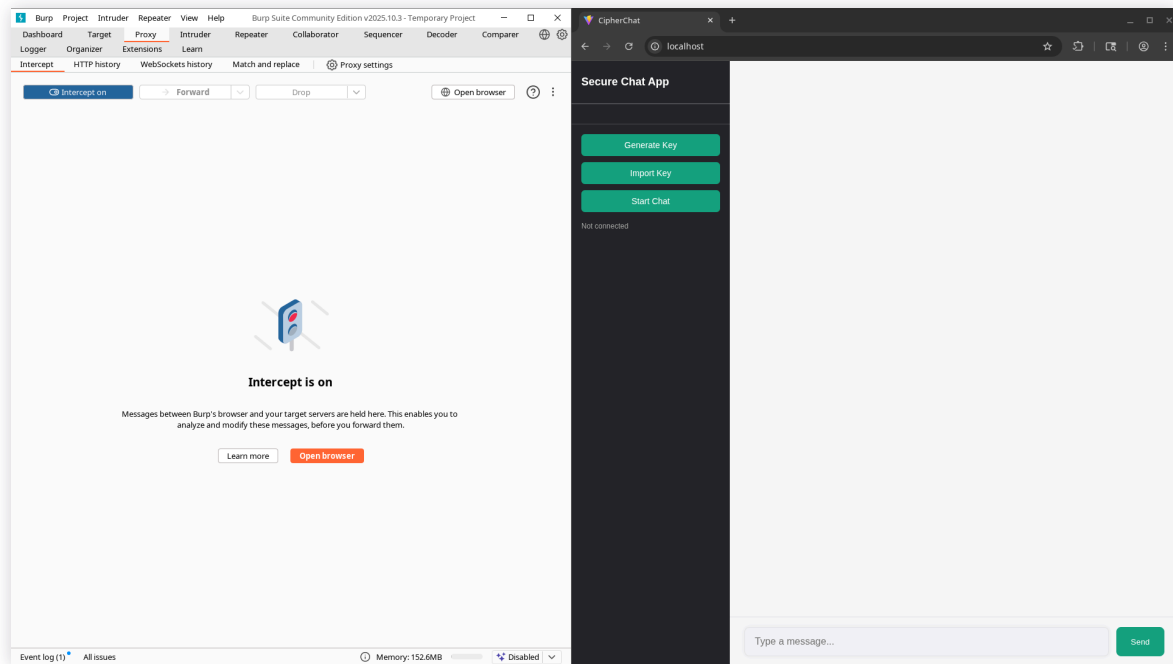


Abbildung 7: Burp: Intercept-Ansicht und Burp-Browser

The screenshot shows the 'HTTP history' tab in Burp Suite. It displays a table of intercepted HTTP requests. The table has columns for #, Host, Method, URL, Params, Edited, Status code, Length, MIME type, and Extension. Two requests are visible: one to http://localhost/ and another to http://localhost/assets/frontend_wasm_bg.wasm, both with a status code of 304.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension
15	http://localhost	GET	/			304	180		
18	http://localhost	GET	/assets/frontend_wasm_bg.wasm			304	182		wasm

Abbildung 8: HTTP History

7.3.3. Initialisierung: Schlüsselgenerierung und Proxy-Authentifizierung

Der Client erzeugt sein Schlüsselpaar und authentifiziert sich gegenüber dem Proxy. Die für die Authentifizierung relevanten Nachrichten (z. B. der initiale Challenge-Austausch und die vom Client gesendete Signatur samt öffentlichem Verifikationsschlüssel) sind in Burp sichtbar. Bei Verwendung des Burp-Browsers erscheinen diese Nachrichten

in der entsprechenden History-Ansicht; bei WebSocket-basierten Handshakes sind die Einträge im WebSockets-History-Tab sichtbar.

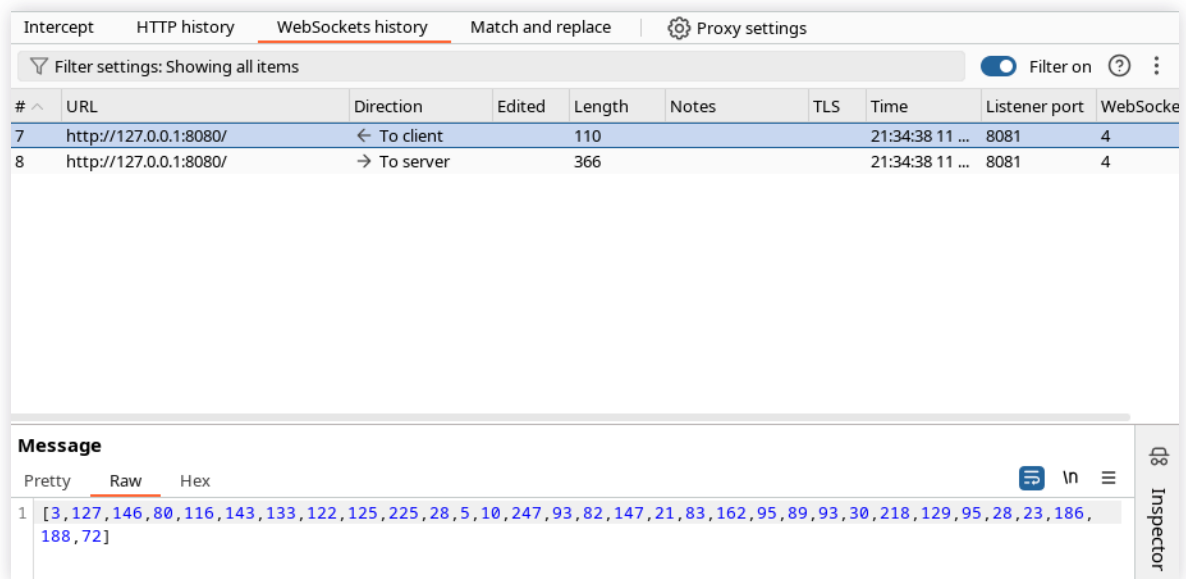


Abbildung 9: Proxy Challenge (Anfrage)

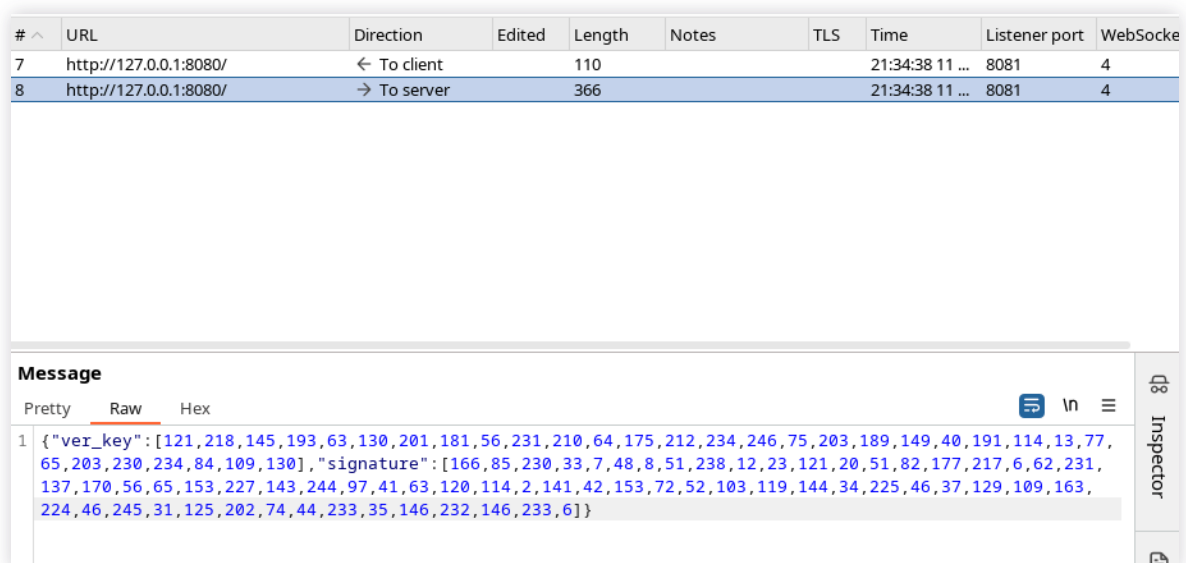


Abbildung 10: Proxy Challenge (Antwort)

7.3.4. Verbindungsaufbau zwischen Clients (SYN / ACK)

Bei Initiierung einer Verbindung zu einem anderen Nutzer wird der SYN-Handshake erzeugt; die anschließende Server-Antwort erfolgt in Form eines ACK-Pakets. Beide Pakettypen (SYN und ACK), inklusive ihrer JSON-serialisierten Felder (z. B. `id`, `client_eph_pub_key`, `syn_digest`, `server_eph_pub_key`), können gesehen werden.

```
Message
Pretty Raw Hex
1 {"SYN":{"id":"9777718657979617271","client_ver_key":[121,218,145,193,63,130,201,181,56,231,210,64,175,212,234,246,75,203,189,149,40,191,114,13,77,65,203,230,234,84,109,130],"server_ver_key":[209,122,223,94,230,127,170,163,194,130,156,121,205,91,245,22,28,107,16,14,74,52,28,32,73,129,49,114,174,138,86,238],"client_eph_pub_key":[48,54,15,15,4,248,234,150,234,84,139,94,57,177,142,166,27,209,217,199,183,198,70,30,29,189,178,202,139,255,109,19],"client_random":[156,37,205,73,220,36,71,142,58,118,192,15,132,195,253,173,74,115,188,94,20,8,240,190,106,196,104,97,106,251,52,221],"timestamp":1762895013595,"signature":[252,237,166,218,251,50,119,187,27,80,224,115,18,233,10,178,101,99,152,52,6,121,71,105,14,250,94,173,172,45,163,23,77,150,248,130,174,50,103,88,189,144,35,124,173,164,137,221,206,178,52,119,80,56,41,181,161,77,123,150,2,199,154,5]}}
```

Abbildung 11: SYN-Paket

```
Message
Pretty Raw Hex
1 {"ACK":{"id":"9777718657979617271","client_ver_key":[121,218,145,193,63,130,201,181,56,231,210,64,175,212,234,246,75,203,189,149,40,191,114,13,77,65,203,230,234,84,109,130],"server_ver_key":[209,122,223,94,230,127,170,163,194,130,156,121,205,91,245,22,28,107,16,14,74,52,28,32,73,129,49,114,174,138,86,238],"syn_digest":[5,242,142,226,228,99,173,140,221,207,34,101,212,102,246,233,111,2,204,7,111,131,73,77,146,188,144,98,254,184,165,28],"server_eph_pub_key":[28,133,216,175,24,249,102,122,237,180,92,102,160,179,58,8,148,16,17,16,170,189,212,215,217,86,2,253,88,214,155,45],"server_random":[147,149,10,35,202,89,99,99,141,210,248,217,207,119,113,136,161,249,106,203,116,252,197,52,22,161,14,167,188,28,27,193],"signature":[136,89,25,42,80,165,15,58,213,195,6,152,80,153,63,240,145,248,96,100,190,22,184,47,82,112,106,179,47,75,226,236,131,92,190,241,192,12,206,251,46,216,85,0,160,246,184,48,178,53,56,139,10,163,52,28,146,129,193,209,139,147,169,1]}}
```

Abbildung 12: ACK-Paket

7.3.5. Laufende verschlüsselte Nachrichten (EncryptedMessage)

Nach erfolgreichem Abschluss des Handshakes werden die Nachrichten im EncryptedMessage-Format ausgetauscht. Diese Einträge lassen sich in der Burp-History als JSON-Objekte identifizieren; die verschlüsselte Nachricht (message), das nonce-Feld und die Metadaten (sender_ver_key, receiver_ver_key, id) sind dort sichtbar und können zwischen Nutzer und Proxy manipuliert werden.

```
Message
Pretty Raw Hex
1 {"EncryptedMessage":{"id":"9777718657979617271","sender_ver_key":[121,218,145,193,63,130,201,181,56,231,210,64,175,212,234,246,75,203,189,149,40,191,114,13,77,65,203,230,234,84,109,130],"receiver_ver_key":[209,122,223,94,230,127,170,163,194,130,156,121,205,91,245,22,28,107,16,14,74,52,28,32,73,129,49,114,174,138,86,238],"message":[82,203,57,250,240,143,138,85,194,26,158,210,51,222,1,31,107,129,170,110,205,16,127,221,229,102,103,64,219,67,187,236,5,136,252,109,147,71,95,128,82,36],"nonce":[31,101,57,135,193,103,58,23,99,100,33,139],"signature":[111,163,101,139,150,6,155,138,153,100,158,95,242,104,180,174,100,102,108,238,132,177,35,212,29,237,143,64,247,40,8,109,52,109,133,10,229,132,52,42,56,173,119,230,206,155,108,100,246,189,0,67,21,236,31,42,171,58,144,126,113,134,148,1]}}
```

Abbildung 13: EncryptedMessage (MSG-Paket)

7.4. SYN im Namen eines fremden Clients senden

In diesem Test wurde geprüft, ob nach erfolgreicher Authentifizierung eines Clients ein Angreifer (hier simuliert durch den Burp-Browser) ein SYN-Paket im Namen eines

anderen, nicht registrierten Clients verschicken kann. Technisch gesagt, ob sich die Sitzungs-/Verbindungslogik durch ein gefälschtes `client_ver_key` austricksen lässt.

Dazu wurde zunächst eine gültige Sitzung eines registrierten Clients aufgebaut und die zugehörigen Handshake-Pakete (inklusive der Authentifizierungsnachricht gegenüber dem Proxy) in Burp aufgezeichnet. Anschließend wurde ein im WebSocket-Verlauf erfasstes SYN-Paket in den Repeater übernommen und dort das Feld `client_ver_key` durch den öffentlichen Schlüssel eines anderen Clients ersetzt, während der restliche JSON-Payload unverändert blieb. Dieses manipulierte Paket wurde erneut an den Proxy gesendet, um zu prüfen, ob die gefälschte Identität akzeptiert wird.

Der Proxy wies das Paket jedoch still ab; es erfolgte keine Bestätigung, kein ACK und keine Weiterleitung zum Ziel-Peer. Daraus lässt sich schließen, dass der Proxy nach der Authentifizierung eine feste Bindung zu nachfolgenden SYN-Anfragen herstellt und gefälschte `client_ver_key`-Felder nicht akzeptiert.

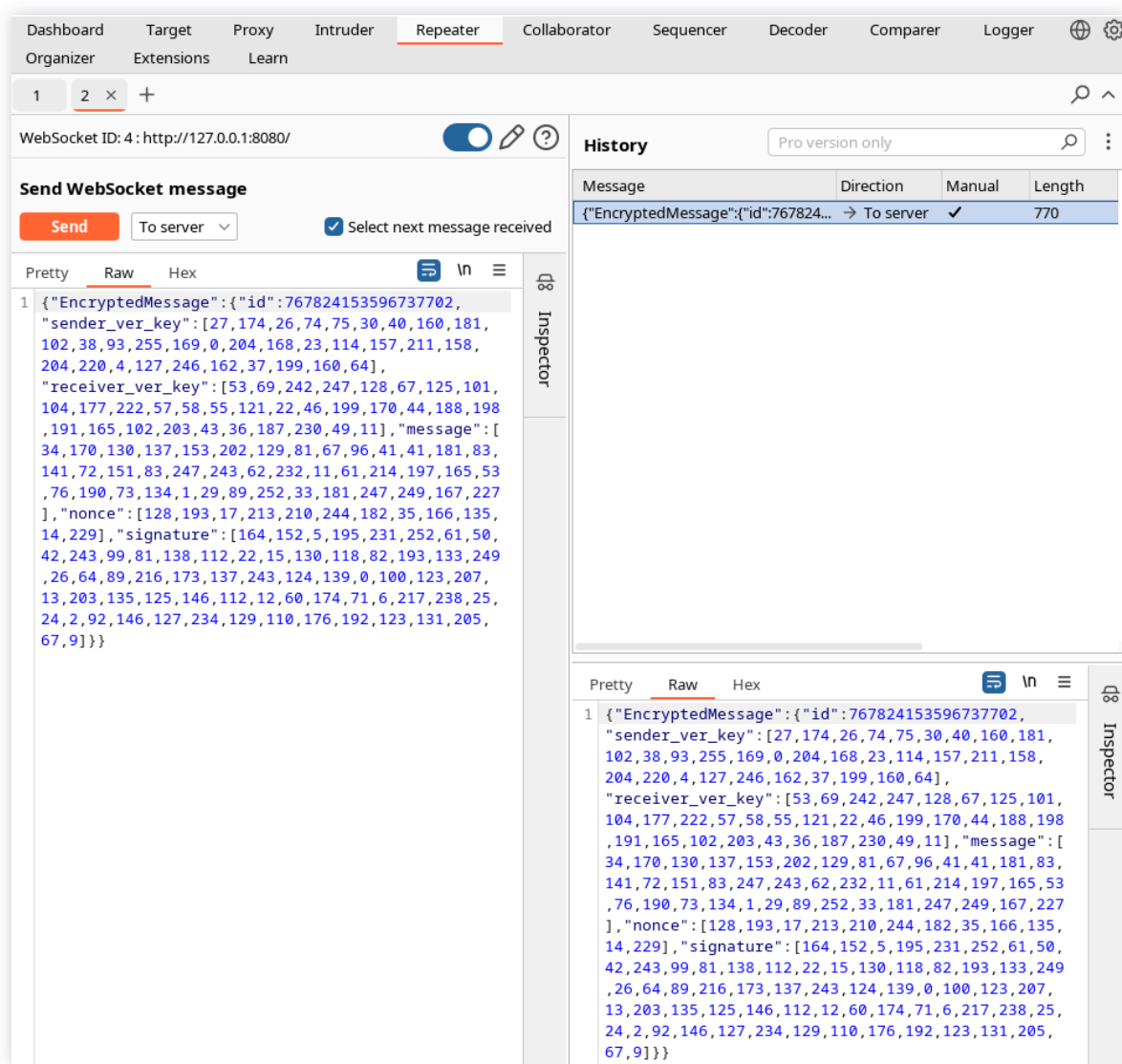


Abbildung 14: Repeater: Wiederholung eines SYN-Pakets mit verändertem `client_ver_key`

7.5. Rolle des Proxys und Anpassung für die Tests

Der Proxy übernimmt einen großen Teil der Sicherheitsprüfung im System, indem er die Signaturen aller eingehenden Pakete verifiziert und dadurch Integrität und Authentizität sicherstellt. Manipulierte Nachrichten werden dadurch bereits abgefangen, bevor sie den Empfänger erreichen.

Für die Protokollanalyse ist diese Schutzwirkung jedoch hinderlich, da so nicht sichtbar wird, wie die Endpunkte selbst auf veränderte oder fehlerhafte Pakete reagieren würden. Es kann jedoch passieren, dass der Proxy selbst kompromittiert wird. Um das Protokoll unabhängig von der Proxy-Sicherheit bewerten zu können, wurde die Signa-

turprüfung im Proxy für alle weiteren Tests deaktiviert. Dadurch konnten auch Angriffe simuliert werden, die ansonsten bereits frühzeitig blockiert worden wären.

7.6. Replay-Angriff auf SYN-Pakete

Hier wurde untersucht, ob Handshake-Nachrichten mit veralteten Zeitstempeln akzeptiert werden. Ein gültiges SYN-Paket wurde abgefangen, einige Sekunden verzögert und erst dann weitergeleitet.

Der Server verwarf den Verbindungsaufbau mit der Meldung: **„tried to start chat ... but timestamp too old“**. Damit zeigt sich, dass der Zeitstempelmechanismus zuverlässig gegen verspätete Handshake-Pakete schützt.

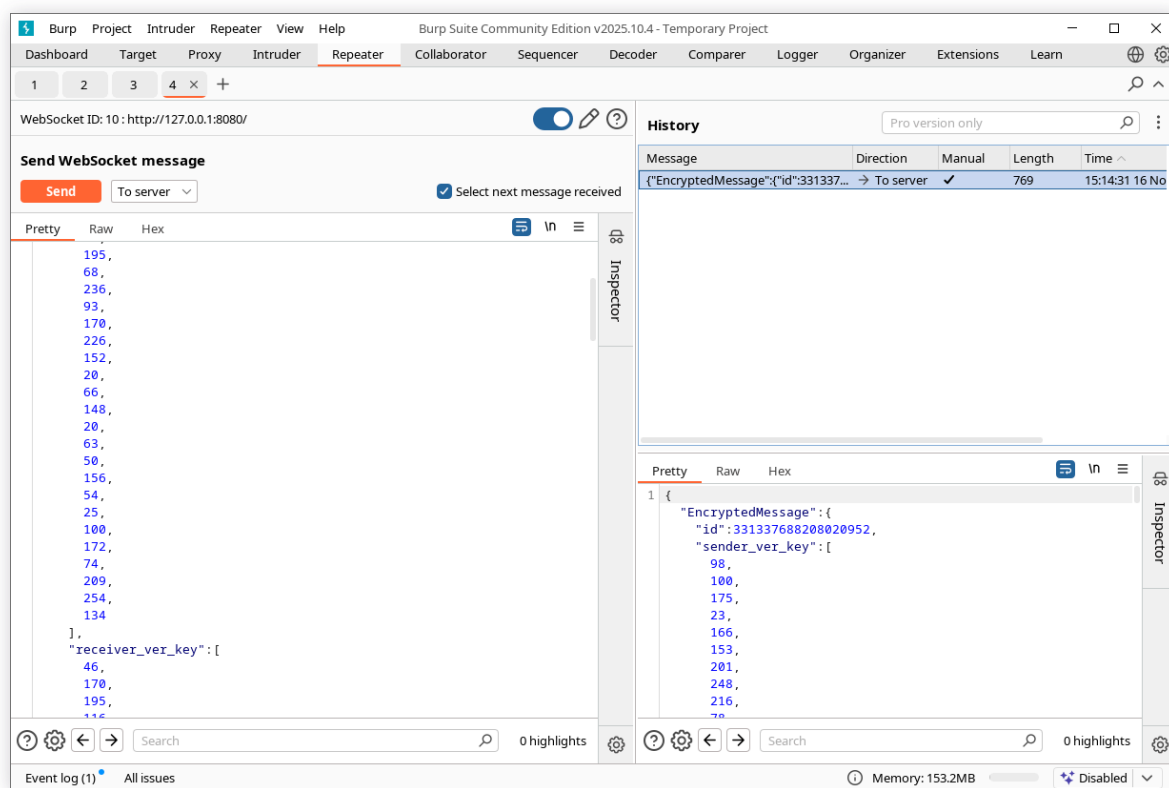


Abbildung 15: Repeater Tab

7.7. Replay-Angriff auf verschlüsselte Nachrichten

In diesem Test wurde geprüft, ob abgefangene, bereits gesendete Nachrichten wieder akzeptiert werden. Dazu wurde eine verschlüsselte Nachricht im Burp-Proxy abgefangen und über den Repeater erneut gesendet.

Der Empfänger erkannte die Wiederholung sofort und wies sie ab: **„attempted to send message with duplicate nonce“**.

Wurde das `nonce` künstlich verändert, schlug die Entschlüsselung fehl:

„attempted to send message but decryption failed: AES decryption failed“.

Damit greifen sowohl die Nonce-Überprüfung als auch die Integritätsprüfung der symmetrischen Verschlüsselung ein.



Abbildung 16: Replay-Angriff Screenshot

7.8. Manipulation des Handshakes (SYN/ACK)

Hier wurde getestet, ob der Handshake durch eine MITM-Manipulation beeinflussbar ist. Ein ACK-Paket wurde im Intercept verändert, indem ein Byte im Payload geändert wurde, und anschließend erneut gesendet.

Der Server lehnte das Paket aufgrund einer ungültigen Signatur ab: **„attempted to send ACK but signature invalid: Verification equation was not satisfied“**. Damit ist bestätigt, dass der Handshake dank kryptografischer Signaturen zuverlässig gegen Manipulation geschützt ist.

7.9. Unautorisierte Nachricht ohne Handshake

Schließlich wurde geprüft, ob verschlüsselte Nachrichten außerhalb einer gültigen Sitzung akzeptiert werden. Dazu wurde eine formal gültige, aber nicht zu einer bestehenden Sitzung gehörende `EncryptedMessage` im Repeater erzeugt und an den Proxy gesendet.

Das Paket wurde abgewiesen, da keine passende Sitzung existierte: **„attempted to send message but chat does not exist“**. Damit wird klar, dass Nachrichten ohne vorausgehenden Handshake technisch nicht akzeptiert werden können.

7.10. Fazit

Die durchgeführten Sicherheitstests zeigen, dass die Schutzmechanismen effektiv gegen diese Angriffsvektoren schützen. Replay-Angriffe werden durch Nonce-Überprüfung und Zeitstempelvalidierung verhindert. Manipulationsversuche während des

Handshakes werden durch Signaturen erkannt und abgewiesen. Zudem akzeptiert die Anwendung keine Nachrichten ohne vorherigen, gültigen Handshake, wodurch unautorisierte Kommunikation unterbunden wird. Insgesamt zeigen die Testergebnisse, dass die Kombination aus Nonces, Signaturen und zeitbasierten Prüfungen einen festen Schutz gegen typische Angriffe bietet und die Sicherheit des End-zu-End-verschlüsselten Protokolls auch in der Praxis gewährleistet.

8. Ergebnisse und Reflexion

8.1. Zielsetzung

Die in Abschnitt 2.2 genannten Ziele wurden eindeutig erreicht: Es wurde ein Chat-App entwickelt, dessen Protokoll Vertraulichkeit, Integrität, Authentizität, Vertraulichkeit und Forward Secrecy ohne sichtbare Sicherheitslücken erreicht.

8.2. Unterschiede zu TLS

Das entwickelte Protokoll basiert grundsätzlich auf TLS 1.2, unterscheidet sich jedoch in mehreren Punkten. Diese Abweichungen sind bewusst gewählt, da eine klassische TLS-Implementierung nicht optimal zu den Anforderungen einer Ende-zu-Ende-verschlüsselten Chat-App passt. Während TLS primär für HTTPS und Webseiten entwickelt wurde, ist dieses Protokoll ausschließlich für eine Chat-App gedacht. Deshalb konnten gewisse Teile des TLS wie Zertifikate weggelassen werden.

8.2.1. X.509-Zertifikate

TLS setzt auf **X.509-Zertifikate**, die eine komplexe **PKI** (Public Key Infrastructure) erfordern. Für eine Chat-Anwendung würde dieser Ansatz unnötigen administrativen Aufwand und zusätzliche Angriffsflächen erzeugen.

Stattdessen verwendet das entwickelte Protokoll reine Ed25519-Schlüssel ohne Zertifikatsschicht, was die Komplexität deutlich reduziert. Da die Vertrauensbeziehungen innerhalb der App ohnehin über benutzerkontrollierte Schlüssel und nicht über DNS-basierte Identitäten geregelt werden, bietet ein Zertifikatsmodell keinen Mehrwert.

8.2.2. Beidseitige Authentifizierung

In TLS ist nur der Server zur Authentifizierung verpflichtet. Client-Zertifikate existieren zwar, kommen in der Praxis aber kaum vor.

Für einen **End-to-End-Chat** ist jedoch zwingend erforderlich, dass beide Kommunikationspartner ihre Identität beweisen. Daher signieren sowohl der Client als auch der Server den gesamten Handshake. Dadurch wird sichergestellt, dass der Proxy keine Rollen vortäuschen oder Nachrichten in fremdem Namen erzeugen kann.

8.2.3. Vereinfachter und minimalistisch gehaltener Handshake

Im Gegensatz zu TLS, das zahlreiche zusätzliche Informationen austauschen muss (**Protokollversionen, Cipher-Suites, Extensions, Kompressionsverfahren, Session-IDs, Tickets** usw.), operiert das hier entwickelte System in einer vollständig **kontrollier-**

ten Umgebung. Dadurch sind **keine Verhandlungen** erforderlich: Es existiert nur **eine Protokollversion**, alle Clients verwenden **denselben Cipher-Suite** (X25519 + Ed25519 + AES-256-GCM + SHA-256) und es gibt **keine optionalen Erweiterungen**. Die Vielfalt klassischer TLS-Kompatibilitätsmechanismen wird somit nicht benötigt.

Dies verhindert Fehlkonfigurationen und reduziert die Angriffsfläche.

8.3. Persönliche Erkenntnisse

Dieses Projekt wies viel mehr Schwierigkeiten auf als ursprünglich geplant. Das Projekt zeigt die Bedeutung der vollständigen Abdeckung aller Sicherheitsaspekte (**Security by Design** statt **Security by Obscurity**), da deren Vernachlässigung Sicherheitslücken einbringen könnte.

8.3.1. Rust und Herausforderungen in der Implementierung

Die Wahl von Rust bot signifikante Vorteile für die Speicher- und Typsicherheit. Die Implementierung war jedoch sehr zeitaufwendig. Ein Beispiel hierfür war die Verwendung komplexer, generischer Typen, wie `GenericArray<u8, U12>` für den Nonce. Dies erforderte eine strikte Versions- und Typenkonsistenz über alle Abhängigkeiten, was die Entwicklungszeit deutlich verlängerte.

8.3.2. Frontend-Integration und Ökosystem-Tools

Die Integration von Rust und Vue.js wurde durch das Tool `wasm-bindgen` erfolgreich realisiert. Die initiale Einrichtung (Setup) war jedoch von Komplexität geprägt. Die verfügbare Dokumentation ist teilweise mangelhaft, was zu einer Häufung unerwarteter Konfigurations- und Kompilierungsfehler führte.

Das Vue.js Framework erwies sich hingegen als sehr stabil und effizient. Die Entwicklung neuer Funktionen konnte mit hoher Geschwindigkeit und wenigen Fehlern durchgeführt werden, was die Produktivität signifikant steigerte.

8.3.3. Tools zur Sicherheitsanalyse und Recherche

Die kostenfreie Version der Burp Suite (Community Edition) besitzt mit den Funktionen „Intercept“ und „Repeater“ alle notwendigen Werkzeuge für die Sicherheitsanalyse und ist sehr benutzerfreundlich.

ChatGPT diente als wertvolle Ausgangsbasis für initiale Recherchen und zur schnellen Validierung von Protokolleigenschaften im Vergleich zu etablierten Standards wie TLS. Es wurde jedoch stets darauf geachtet, alle generierten Informationen kritisch zu prüfen und durch anerkannte Quellen zu verifizieren.

8.3.4. Backend-Entwicklung

Die Backend-Entwicklung unter Verwendung des asynchronen Runtimes Tokio stellte die größte technische Hürde dar. Die Komplexität konnte schließlich durch die Adaption einer ähnlichen Proxy-Implementierungsvorlage überwunden werden, was eine verhältnismäßig zügige Finalisierung des Backends ermöglichte.

9. Einsatzmöglichkeiten und Ausblick

Die entwickelte Chat-Applikation bietet bereits jetzt die wichtigsten Kommunikationsfunktionen. Die Hauptanwendung liegt im direkten Austausch von Nachrichten zwischen Benutzern. Ein besonderer Fokus liegt auf dem sicheren Austausch von sensiblen Informationen, wie etwa Passwörtern, da die Kommunikationswege entsprechend geschützt sind.

Über die erreichten Ziele und den aktuellen Umfang dieser Arbeit hinaus eröffnen sich verschiedene theoretische Weiterentwicklungsmöglichkeiten für die Anwendung. Diese potenziellen Funktionen skizzieren das langfristige Potenzial des entwickelten Systems, sind jedoch explizit nicht Teil der Zielsetzung dieser Arbeit und wären Gegenstand zukünftiger Projekte:

- **Effizientere Verbindungsverwaltung:** Es wäre technisch möglich, Mechanismen zu implementieren, die es der Anwendung erlauben, bestehende Verbindungen fortzusetzen und wiederherzustellen, anstatt nach einer Unterbrechung stets einen neuen Verbindungsaufbau initiieren zu müssen.
- **Vereinfachte Schlüsselmanagement:** Der Austausch der kryptografischen Schlüssel könnte durch eine einfachere Methode optimiert werden. Anstatt die 32 Bytes manuell zu teilen, könnte dies beispielsweise über die Generierung und das Scannen eines QR-Codes erfolgen.
- **Erweiterung um Gruppenchats:** Eine denkbare funktionale Erweiterung ist die Implementierung von Gruppenchats, um die Kommunikation mehrerer Teilnehmer in einem gemeinsamen Kanal zu ermöglichen.
- **Horizontale Skalierung:** Im Hinblick auf Skalierbarkeit und Hochverfügbarkeit wäre die theoretische Unterstützung von mehreren Servern notwendig, um eine potenziell größere Benutzerbasis und eine bessere Lastverteilung zu gewährleisten.

10. Anhang

Um die praktische Umsetzung dieser Arbeit sowie den Entstehungsprozess nachvollziehbar zu machen, wurden alle relevanten Ressourcen online zur Verfügung gestellt. Unter der folgenden URL findet sich eine begleitende Website, die als zentraler Einstiegspunkt dient:

<https://aba.antonaparin.com>

Diese Webseite enthält Verweise auf die folgenden Komponenten:

- **Live-Demo der Chat-App:** Über die Website kann die entwickelte Applikation direkt aufgerufen und in einer Live-Umgebung getestet werden.
- **Zentrales GitHub-Repository:** Hier findet sich der vollständige Quellcode des gesamten Projekts. Dies umfasst:
 - Den Programmcode der modernen Chat-App (Frontend/Backend).
 - Den Typst-Quellcode, aus dem diese schriftliche Arbeit generiert wurde.
- **PDF-Version der Arbeit:** Für den schnellen Zugriff ist die bereits kompilierte PDF-Datei der Arbeit ebenfalls auf der Website hinterlegt.



Abbildung 17: <https://aba.antonaparin.com> QR-Code

Literaturverzeichnis

- [1] Cloudflare, „What Is Transport Layer Security?“, Zugriffen: 25. September 2025. [Online]. Verfügbar unter: <https://www.cloudflare.com/learning/ssl/transport-layer-security-tls/>
- [2] C. Kaufman, R. Perlman, M. Speciner, und R. Perlner, *Network Security: Private Communication in a Public World*, 3. Aufl. Hoboken, NJ: Pearson Education, 2022. Zugriffen: 5. September 2025. [Online]. Verfügbar unter: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=934860
- [3] D. Upadhyay, N. Gaikwad, M. Zaman, und S. Sampalli, „Investigating the avalanche effect of various cryptographically secure hash functions and hash-based applications“, *IEEE Access*, Bd. 10, S. 112472–112486, Okt. 2025.
- [4] „Transport Layer Security (TLS) - Security | MDN“. Zugriffen: 9. Februar 2026. [Online]. Verfügbar unter: https://developer.mozilla.org/en-US/docs/Web/Security/Defenses/Transport_Layer_Security
- [5] S. Klabnik, C. Nichols, und C. Krycho, „The Rust Programming Language“. Zugriffen: 30. September 2025. [Online]. Verfügbar unter: <https://doc.rust-lang.org/book/>
- [6] „Introduction - Rust and WebAssembly“. Zugriffen: 9. Februar 2026. [Online]. Verfügbar unter: <https://rustwasm.github.io/book/>
- [7] E. You, „Vue.js“. [Online]. Verfügbar unter: <https://vuejs.org/>
- [8] „WebSocket“. [Online]. Verfügbar unter: <https://developer.mozilla.org/en-US/docs/Web/API/WebSocket>
- [9] Cloudflare, „What Happens in a TLS Handshake?“, Zugriffen: 25. September 2025. [Online]. Verfügbar unter: <https://www.cloudflare.com/learning/ssl/what-happens-in-a-tls-handshake/>
- [10] Z. Fedotkin, „WebSocket Turbo Intruder: Unearthing the WebSocket Goldmine“. Zugriffen: 12. November 2025. [Online]. Verfügbar unter: <https://portswigger.net/research/websocket-turbo-intruder-unearthing-the-websocket-goldmine>

Abbildungsverzeichnis

Abbildung 1	TLS 1.2 Handshake	12
Abbildung 2	Kommunikationsfluss Gesamtbild	17
Abbildung 3	Kommunikationsfluss Frontend	17
Abbildung 4	Protokollentwurf	18
Listing 1	SYN Struktur	20
Listing 2	ACK Struktur	21
Listing 3	EncryptedMessage Struktur	22
Listing 4	DecryptedMessagePayload Struktur	22
Listing 5	Fehlerbehandlung EncryptedMessage Beispiel	23
Listing 6	AuthPacket Struktur	24
Listing 7	PeerMap Typdefinition	24
Listing 8	Proxy Weiterleitung	25
Listing 9	Proxy main-Funktion	26
Listing 10	Rust #[wasm_bindgen] Beispiel	27
Listing 11	JavaScript WASM Import Beispiel	27
Listing 12	Frontend-WASM Zustand	28
Listing 13	Zustand Zugriff Beispiel	28
Listing 14	proxy_connect Implementation	30
Listing 15	read_packet Signatur	31
Listing 16	handle_syn FunktionsSignatur Beispiel	31
Listing 17	handle_encrypted_message Prüfung Beispiel	34
Listing 18	Nachrichtenentschlüsselung	35
Listing 19	Einfügen und Weiterleiten einer empfangenen Nachricht	35
Listing 20	Zähler Erhöhung	35
Listing 21	AES-GCM-Verschlüsselung	36
Listing 22	Paket-Serialisierung	36
Abbildung 5	Senden Überblick	37
Listing 23	store.js Zustand	39
Abbildung 6	Screenshot der Chat-App	41
Listing 24	Backend Dockerfile	42
Listing 25	Frontend Dockerfile	43
Listing 26	docker-compose.yml	43
Tabelle 1	Empfang von Proxy	44
Tabelle 2	Authentifizierungsanfrage an den Proxy	45

Tabelle 3	SYN	45
Tabelle 4	ACK	46
Tabelle 5	EncryptedMessage	47
Abbildung 7	Burp: Intercept-Ansicht und Burp-Browser	49
Abbildung 8	HTTP History	49
Abbildung 9	Proxy Challenge (Anfrage)	50
Abbildung 10	Proxy Challenge (Antwort)	50
Abbildung 11	SYN-Paket	51
Abbildung 12	ACK-Paket	51
Abbildung 13	EncryptedMessage (MSG-Paket)	51
Abbildung 14	Repeater: Wiederholung eines SYN-Pakets mit verändertem client_ver_key	53
Abbildung 15	Repeater Tab	54
Abbildung 16	Replay-Angriff Screenshot	55
Abbildung 17	https://aba.antonaparin.com QR-Code	61

Alle Abbildungen, Listings und Tabellen stammen von **Anton Aparin**